
Documentação de Projeto

para o sistema

Gerenciamento de Inadimplentes

Versão 1.4

Projeto de sistema elaborado pelo aluno Matheus Pereira e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo do professor Cleiton Silva Tavares, Danilo de Quadros Mais Filho , Leonardo Vilela Cardoso e Raphael Ramos Dias Costas, orientação acadêmica do professor Cleiton e orientação de TCC II do professor (a ser definido no próximo semestre).

30/11/2025

Tabela de Conteúdo

1. Introdução	1
2. Modelo de Usuário e Requisitos	1
2.1 Descrição de Atores	2
2.2 Modelo de Casos de Uso	4
2.3 Modelo de Casos de Uso e Histórias de Usuários	5
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	9
3. Modelos de Projetos	9
3.1 Diagrama de Classe	10
3.2 Diagrama de Sequência	12
3.3 Diagramas de Comunicação	15
3.4 Arquitetura	19
3.5 Diagrama de Estado	21
3.6 Diagrama de Componentes e Implantação	22
4. Projeto de Interface do Usuário	24
5. Glossário e Modelos de Dados	29
6. Casos de Testes	33
7. Cronograma e Processos de Implantação	45

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Matheus Pereira	28/09/25	Preenchimento das seções 1 até 2.1	1.0
Matheus Pereira	09/10/25	Preenchimento das seções 2.1,2,2.2,2.3,3.1, 3.2, 3.3	1.1
Matheus Pereira	02/11/25	Preenchimento das seções 3.4,3.5,3.6 e 5	1.2
Matheus Pereira	18/11/25	Preenchimento das seções 6 e 7	1.3
Matheus Pereira	30/11/25	Atualização das seções 6 e 7	1.4

1. Introdução

O presente documento contempla a elaboração e a revisão de modelos de domínio, bem como a definição de modelos de projeto referentes ao Sistema de Gerenciamento de Inadimplentes. Trata-se de uma aplicação voltada ao controle e acompanhamento de clientes inadimplentes, permitindo o registro de débitos, o monitoramento de pagamentos em atraso e a emissão de relatórios gerenciais que apoiam a tomada de decisão. Tais modelos têm como objetivo fornecer uma representação estruturada dos principais elementos do sistema, suas interações e responsabilidades, subsidiando, assim, o processo de desenvolvimento da aplicação de maneira consistente e alinhada aos requisitos previamente levantados.

2. Modelos de Usuário e Requisitos

Esta seção tem como objetivo descrever os usuários e atores do sistema, assim como os requisitos aos quais esse deve atender. Para isso, é apresentada uma breve descrição de cada ator, assim como um modelo de caso de uso e as histórias de usuários relacionadas, ambos que servem de referência para o desenvolvimento do sistema. Por último, são apresentados o diagrama de sequência do sistema.

2.1 Descrição de Atores

Nesta subseção, são apresentados os atores que interagem com o Sistema de Gerenciamento de Inadimplentes. Cada ator possui responsabilidades específicas, que estão diretamente relacionadas ao uso e às funcionalidades disponibilizadas pela aplicação.

Responsável Financeiro

O Responsável Financeiro constitui-se como o principal usuário do sistema. Compete a este ator o cadastramento de clientes inadimplentes, o registro de pagamentos realizados, a atualização das informações financeiras e a emissão de relatórios gerenciais. Além disso, é incumbido de efetuar a autenticação no sistema, garantindo a segurança do acesso às informações. Sua participação é essencial para assegurar a confiabilidade dos dados e a efetividade do controle da inadimplência, uma vez que concentra a maior parte das operações críticas do sistema.

Proprietária do Escritório de Contabilidade

A Proprietária do Escritório de Contabilidade interage com o sistema de forma complementar, assumindo um papel de caráter gerencial. Suas atividades concentram-se na consulta a clientes inadimplentes, na visualização de relatórios financeiros e na análise de indicadores disponibilizados em dashboards. A partir destas informações, a proprietária obtém uma visão consolidada do cenário financeiro do escritório, o que subsidia a definição de estratégias de cobrança e o planejamento administrativo.

2.2 Modelos de Usuários

Nesta seção são apresentadas as personas que representam os principais usuários do Sistema de Gerenciamento de Inadimplentes. As personas foram elaboradas a partir da análise do contexto do escritório de contabilidade, dos processos de negócio e das necessidades identificadas no levantamento de requisitos.

A Tabela 1 descreve a persona do usuário Cláudia Pereira e Silva, proprietária e gestora administrativa do escritório. Nela é possível identificar que Cláudia sente falta de relatórios claros e consolidados, o que dificulta a identificação rápida dos clientes que mais possuem dívidas. Também é perceptível que ela busca acompanhar indicadores de inadimplência e ter uma visão consolidada da situação financeira. Por fim, após análise é possível ver que Cláudia deseja obter relatórios periódicos, dashboards com indicadores financeiros e uma visão geral da inadimplência que apoie suas decisões estratégicas.

Persona 1: Claudia Pereira	
Idade:	55 anos
Cargo:	Proprietária e gestora administrativa
Objetivos:	Acompanhar indicadores de inadimplência, ter uma visão consolidada da situação financeira e apoiar decisões estratégicas do escritório.
Frustrações:	Falta de relatórios claros e consolidados; dificuldade de identificar rapidamente os clientes que mais devem.
Necessidades:	Obter relatórios periódicos, dashboards com indicadores financeiros e visão geral da inadimplência.

Tabela 1. Persona Claudia Pereira.

A Tabela 2 descreve a persona do usuário José Carlos, responsável pelo setor financeiro. Nela é possível identificar que José, apesar de ser experiente na área financeira, ainda depende de anotações manuais em cadernos, o que aumenta o risco de falhas e dificulta a atualização em tempo real dos registros. Também é perceptível que ele busca reduzir erros manuais e gerar relatórios confiáveis para apoiar a gestão do escritório. Por fim, após análise é possível ver que José deseja um sistema simples, seguro e de fácil usabilidade, que permita cadastrar inadimplentes, registrar pagamentos e emitir relatórios precisos.

Persona 2: José Carlos	
Idade:	55 anos
Cargo:	Responsável pelo setor financeiro
Objetivos:	Manter controle atualizado sobre clientes inadimplentes; reduzir erros manuais; gerar relatórios para apoiar a gestão.
Frustrações:	Atualmente depende de anotações em cadernos, o que aumenta o risco de falhas, dificulta a atualização em tempo real e compromete a precisão dos registros.
Necessidades:	Sistema simples, seguro e de fácil usabilidade, que permita cadastrar inadimplentes, registrar pagamentos e emitir relatórios confiáveis.

Tabela 2. Persona José Carlos.

2.3 Modelo de Casos de Uso e Histórias de Usuários

Esta subseção tem como objetivo descrever os casos de uso e as histórias de usuário previstos para o projeto. Para tanto, apresenta-se um diagrama de casos de uso em que todos são listados. De forma complementar, também são apresentadas as histórias de usuário relacionadas às funcionalidades previstas para o sistema a ser desenvolvido.

2.3.1 Diagrama de caso de uso

O diagrama de casos de uso (figura 1) representa as funcionalidades do sistema financeiro do escritório, com dois atores principais: o Responsável Financeiro e a Proprietária do Escritório. O Responsável Financeiro realiza operações como registro de pagamentos, consulta e cadastro de clientes inadimplentes, geração de relatórios, exportação de dados, autenticação e visualização de indicadores. A ação de registrar pagamento inclui automaticamente a atualização do saldo devedor. Já a Proprietária do Escritório possui acesso à visualização do dashboard e à geração de relatórios, com foco em análise e tomada de decisão.

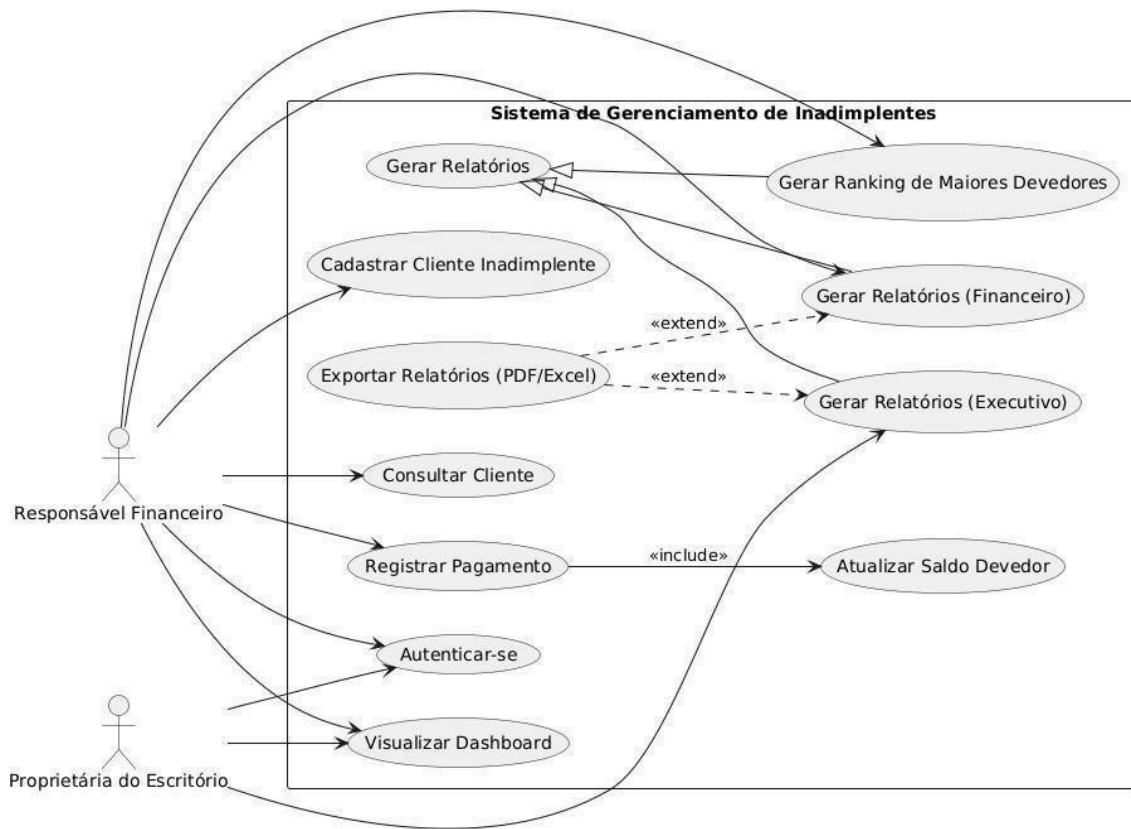


Figura 1. Diagrama de Caso de Uso

2.3.2 Histórias do Usuário

Nesta seção são listadas as histórias de usuários elucidadas para o sistema financeiro proposto. Para fins de organização, utiliza-se o identificador no formato US#ID, em que US refere-se a *User Story*.

As histórias de usuários identificadas para o sistema são:

US1. Como responsável financeiro, gostaria de registrar pagamentos de clientes para manter o controle atualizado das pendências.

US2. Como responsável financeiro, eu gostaria de atualizar automaticamente o saldo devedor ao registrar um pagamento, para garantir consistência nas informações.

US3. Como responsável financeiro, eu gostaria de cadastrar clientes inadimplentes para controlar melhor os casos de atraso.

US4. Como responsável financeiro, eu gostaria de gerar relatórios financeiros para análise interna.

US5. Como responsável financeiro, eu gostaria de exportar relatórios em PDF ou Excel para facilitar o compartilhamento e arquivamento.

US6. Como responsável financeiro, eu gostaria de visualizar o *ranking* de maiores devedores para identificar os principais inadimplentes.

US7. Como responsável financeiro, eu gostaria de consultar dados dos clientes para facilitar o atendimento e a tomada de decisões.

US8. Como responsável financeiro, eu gostaria de gerenciar autenticação para garantir o acesso seguro ao sistema.

US9. Como proprietária do escritório, eu gostaria de visualizar um *dashboard* com indicadores financeiros para acompanhar a saúde do negócio.

US10. Como proprietária do escritório, eu gostaria de gerar relatórios sempre que necessário para entender o desempenho financeiro da empresa.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Esta seção descreve os principais fluxos de interação do SGI – Sistema de Gerenciamento de Inadimplentes por meio de diagramas de sequência, detalhando a ordem das mensagens entre App (usuário/Responsável Financeiro), Backend (API) e serviços de domínio. Cada fluxo possui um Contrato de Operação com pré-condições, pós-condições e exceções. As legendas e referências seguem o padrão do documento.

O fluxo de autenticação inicia quando o usuário informa suas credenciais no app; o backend delega a verificação ao AuthService, que consulta o *UsuarioRepository*; com credenciais válidas, o serviço emite o tokenSessao e identifica o perfil; por fim, o backend retorna 200 OK ao app com o token e o perfil do usuário.

Contrato	Autenticar Usuário
Operação	autenticar(telefone: String, senhaOuOTP: String) -> token: String, perfil: Perfil
Referências cruzadas	US8 – Gerenciar autenticação (acesso seguro ao sistema)
Pré-condições	App online; credenciais informadas; usuário existente/ativo.
Pós-condições	Sessão ativa com tokenSessao válido; perfil associado; auditoria “login_sucesso”.

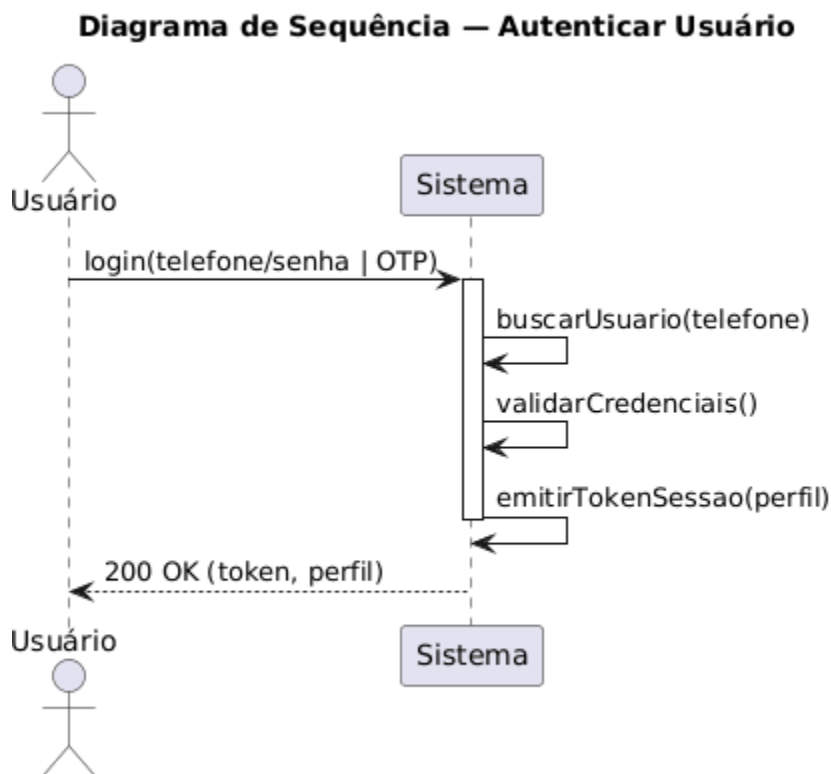


Figura 2 - Autenticar Usuario

Conforme será mostrado na Figura 3. O Responsável Financeiro seleciona o cliente e envia os dados do pagamento; no backend, o `PagamentoService` é acionado para recuperar o cliente, persistir o pagamento e, no mesmo fluxo transacional, recalculando o saldo devedor no repositório; ao final, a API retorna ao app o recibo e o novo saldo confirmado.

Contrato	Registrar Pagamento
Operação	<code>registrarPagamento(clienteId: UUID, valor: Money, data: Date, comprovante: Blob) -> recibo: PDF, novoSaldo: Money</code>
Referências cruzadas	US1 e US2
Pré-condições	Usuário autenticado; <code>clienteId</code> existente/ativo; <code>valor > 0</code> ; <code>data ≤ hoje</code> .
Pós-condições	Pagamento persistido; saldo recalculado; recibo emitido; auditoria “pagamento_registrado”.

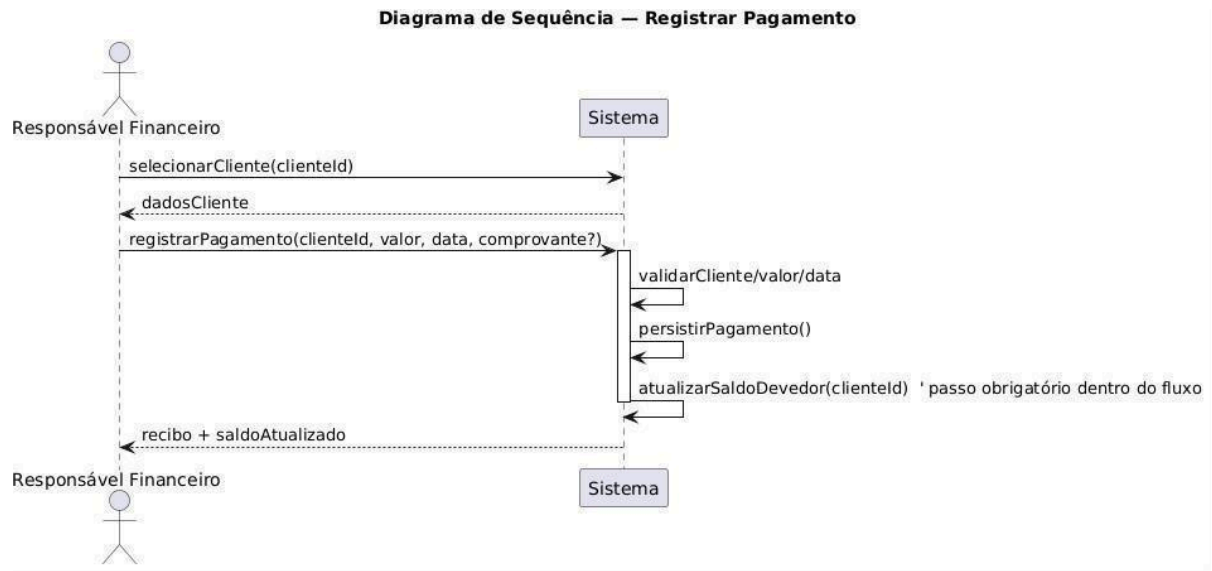


Figura 3 - Registrar Pagamento

Conforme será mostrado na Figura 4, para um cliente previamente validado o responsável financeiro registra uma nova inadimplência informando valor, vencimento e descrição; o InadimplenciaService aplica as regras de negócio, valida os dados e persiste o lançamento com status inicial “Em Aberto”, gerando o respectivo protocolo; em seguida, o backend confirma ao aplicativo a criação do registro (incluindo o identificador/protocolo) e registra a trilha de auditoria do evento, com data-hora, usuário e origem da operação.

Contrato	Registrar Inadimplência
Operação	registrarInadimplencia(clienteId: UUID, valor: Money, vencimento: Date, descricao: String) -> protocolo: String
Referências cruzadas	US3
Pré-condições	clienteId existente/ativo; valor > 0; vencimento ≥ hoje; usuário autenticado.
Pós-condições	Inadimplência criada (status = EmAberto); auditoria “inadimplencia_registrada”.

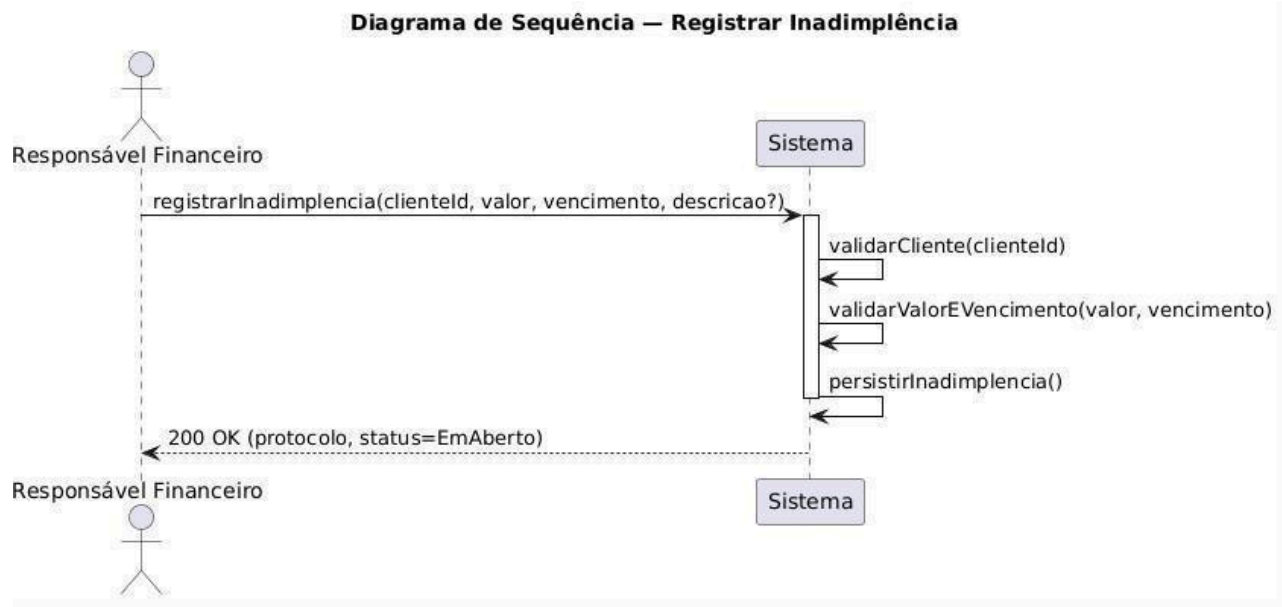


Figura 4 — Registrar Inadimplência.

Conforme será mostrado na Figura 5, o usuário solicita a geração de um relatório (por exemplo, ranking de maiores devedores ou resumo por período), parametrizando os filtros desejados; o `RelatorioService` consulta o repositório, consolida os dados e retorna o dataset/visão para exibição no aplicativo; opcionalmente, o usuário pode acionar a exportação em PDF ou Excel, gerando o arquivo correspondente para download e arquivamento.

Contrato	Gerar Relatório
Operação	<code>gerarRelatorio(tipo: Enum, periodo: {ini, fim}, filtros?: Map) -> view/dataset</code>
Referências cruzadas	US4,US5,US6
Pré-condições	Usuario autenticado; filtros válidos.
Pós-condições	Resultado exibido; se exportado, arquivo gerado (PDF/Excel) e link disponibilizado

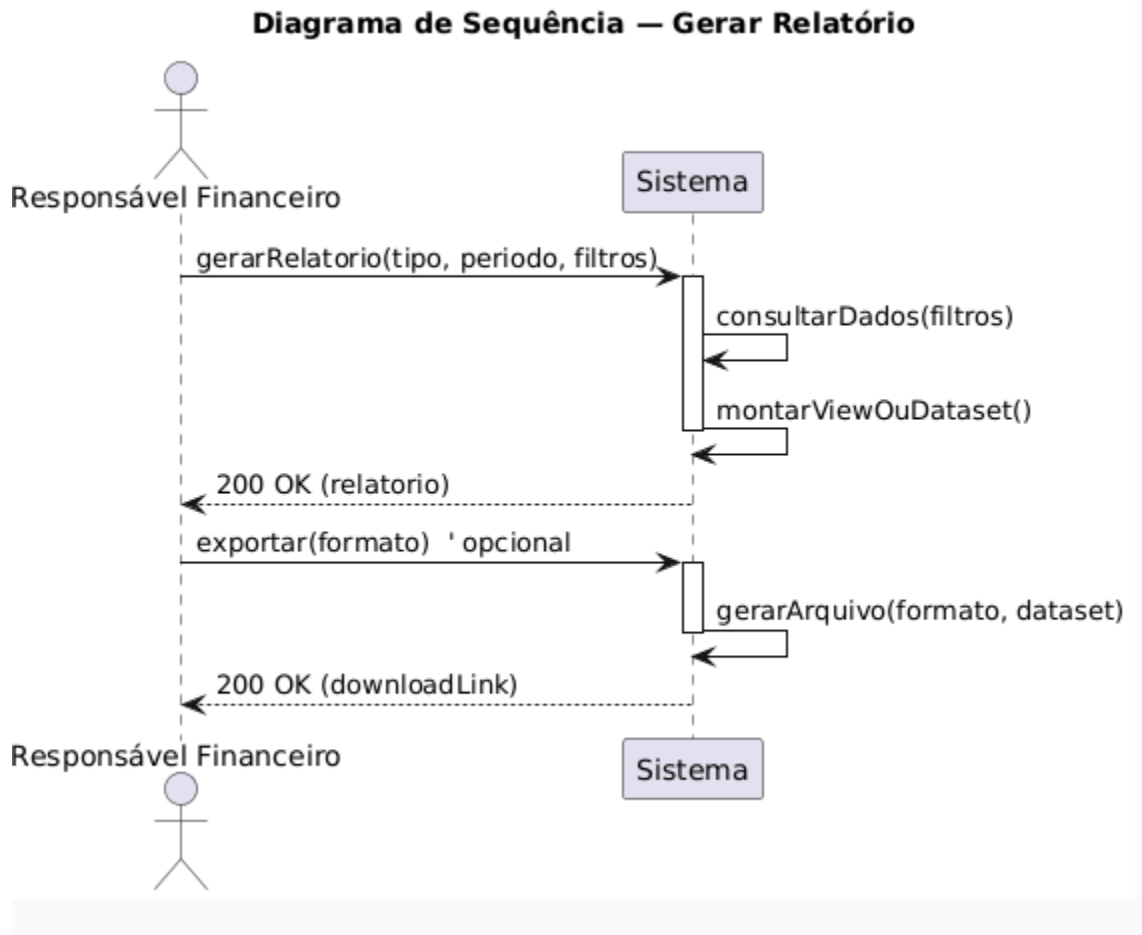


Figura 5 — Gerar Relatório.

Conforme será mostrado na Figura 6, o fluxo de recálculo de saldo consolida os débitos em aberto e os pagamentos efetivados de um cliente, podendo ser acionado manualmente pela interface do usuário ou automaticamente como processo interno subsequente à confirmação de pagamento; ao término da execução, o novo saldo é persistido no repositório de dados e retornado ao aplicativo, assegurando a consistência entre os lançamentos registrados e a visão financeira apresentada ao usuário.

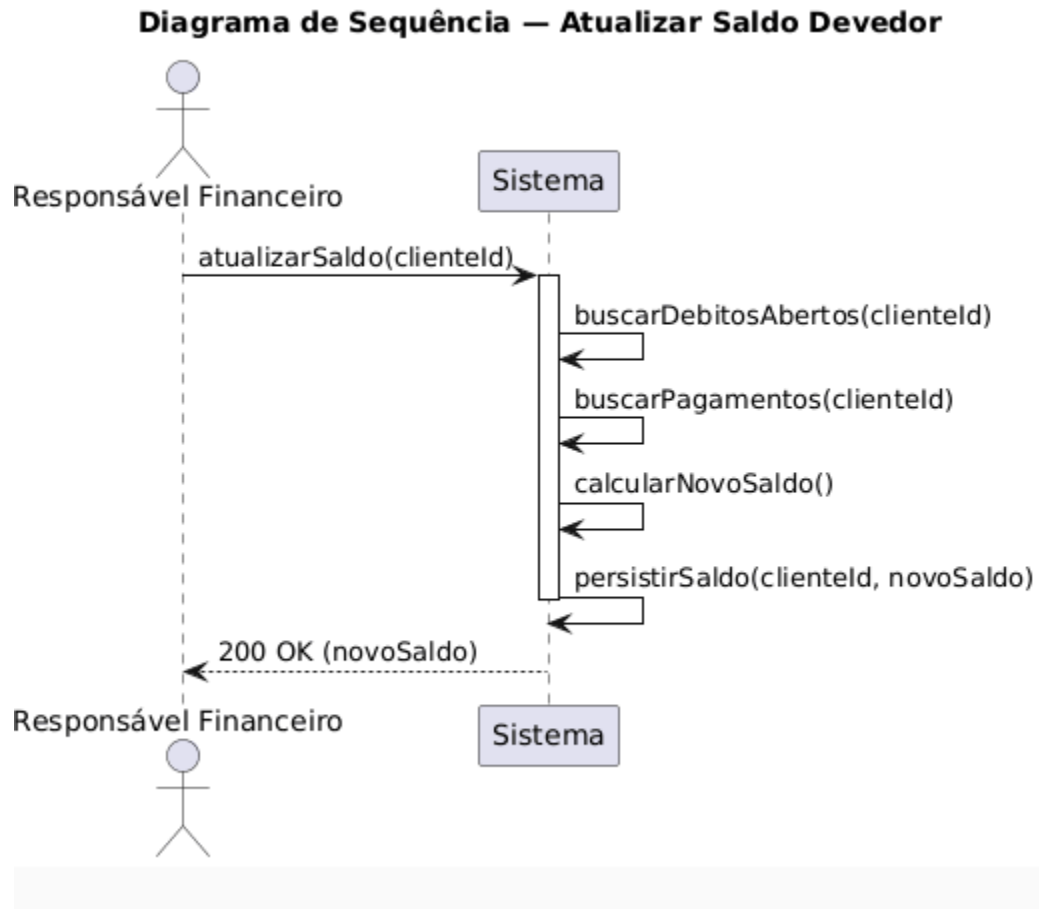


Figura 6 — Atualizar saldo devedor

Contrato	Atualizar Saldo Devedor
Operação	atualizarSaldoDevedor(clientId)
Referências cruzadas	US2, atualizar saldo devedor.
Pré-condições	Usuário autenticado (na execução manual); <i>clientId</i> válido.
Pós-condições	novoSaldo calculado e persistido; auditoria “saldo_recalculado”.

3. Modelos de Projeto

Esta seção apresenta os modelos de projeto do sistema proposto, organizados nas seguintes subseções: Visão Geral dos Modelos (Seção 3.1), Diagrama de Classes (Seção 3.2), Diagramas de Sequência (Seção 3.3), Diagrama de Comunicação (Seção 3.4), Arquitetura (Seção 3.5), Diagramas de Estados (Seção 3.6) e Diagramas de Componentes e Implantação (Seção 3.7). Em conjunto, esses artefatos oferecem uma visão integrada das estruturas, dos comportamentos e da distribuição do sistema.

3.1 Diagrama de Classes

A Figura 6 apresenta o diagrama de classes do núcleo de domínio do sistema de gerenciamento de inadimplentes, destacando as entidades centrais, seus principais atributos e relacionamentos. A classe Cliente representa pessoas físicas ou jurídicas cadastradas, com dados básicos e um atributo status que indica se está adimplente ou inadimplente, conforme o enum StatusCliente. A classe Divida representa os títulos em aberto de cada cliente, contendo informações como valor, vencimento e status definido pelo enum StatusDivida (EM_ABERTO, PARCIAL, QUITADA ou VENCIDA), além de métodos para registrar pagamentos e atualizar saldos. Já a classe Pagamento registra valores, datas e métodos de pagamento, permitindo múltiplos pagamentos por dívida, contemplando liquidações parciais ou totais.

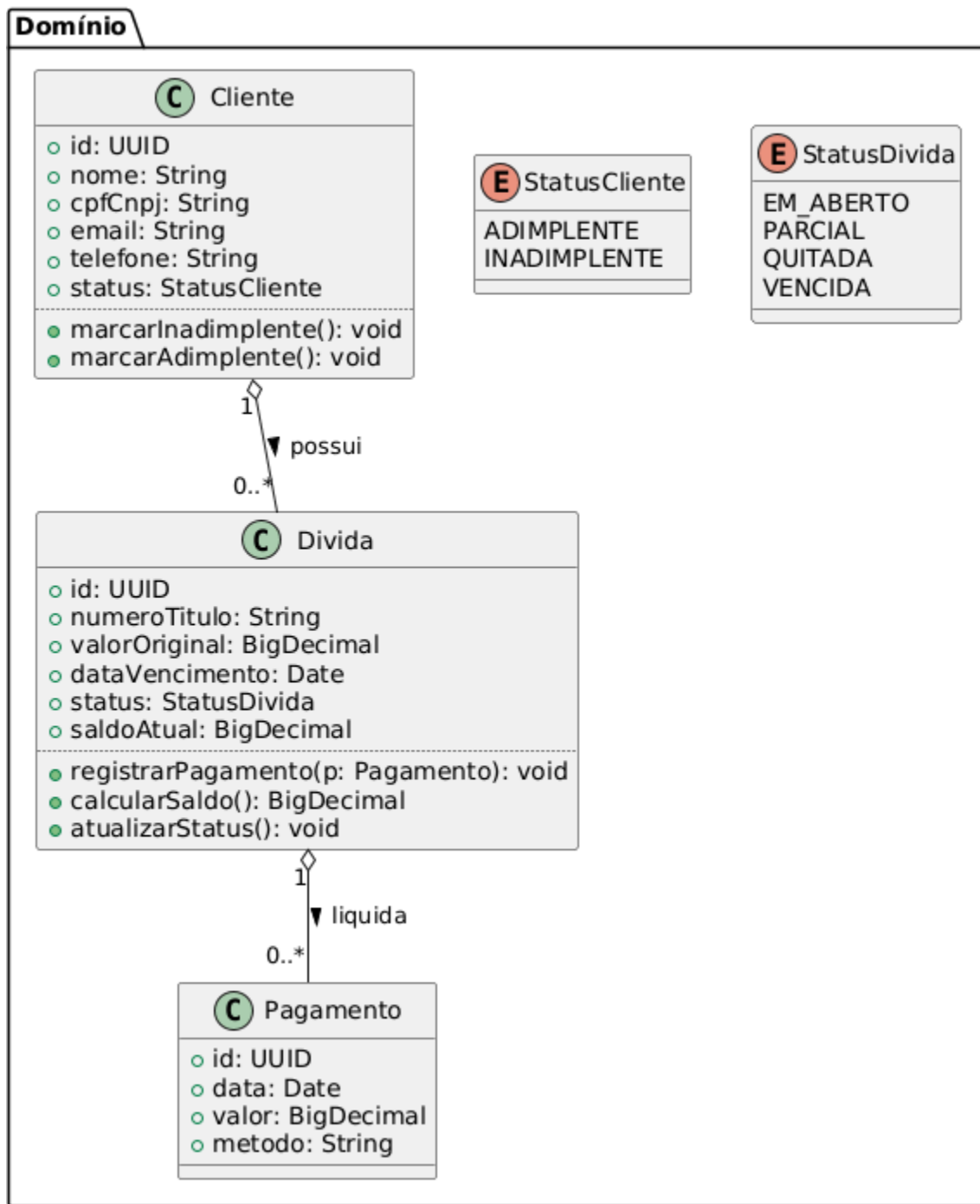


Figura 7 — Diagrama de Classes

3.2 Diagramas de Sequência

A Figura 7 apresenta o diagrama de sequência referente ao fluxo principal de registro de pagamento no sistema de gerenciamento de inadimplentes. Ele descreve a interação entre o Responsável Financeiro, a Aplicação Web, o Serviço de Cobrança e os repositórios de dados. O processo inicia com a solicitação de registro de pagamento, seguida da validação dos dados e da verificação do status da dívida. Caso os dados sejam inválidos ou a dívida esteja quitada, uma mensagem de erro é exibida ao usuário. Se estiver tudo correto, o sistema registra o pagamento, atualiza o saldo e o status da dívida, e ajusta o status do cliente conforme a situação financeira. Ao final, uma confirmação com o novo saldo é retornada ao responsável, garantindo consistência e rastreabilidade da operação.

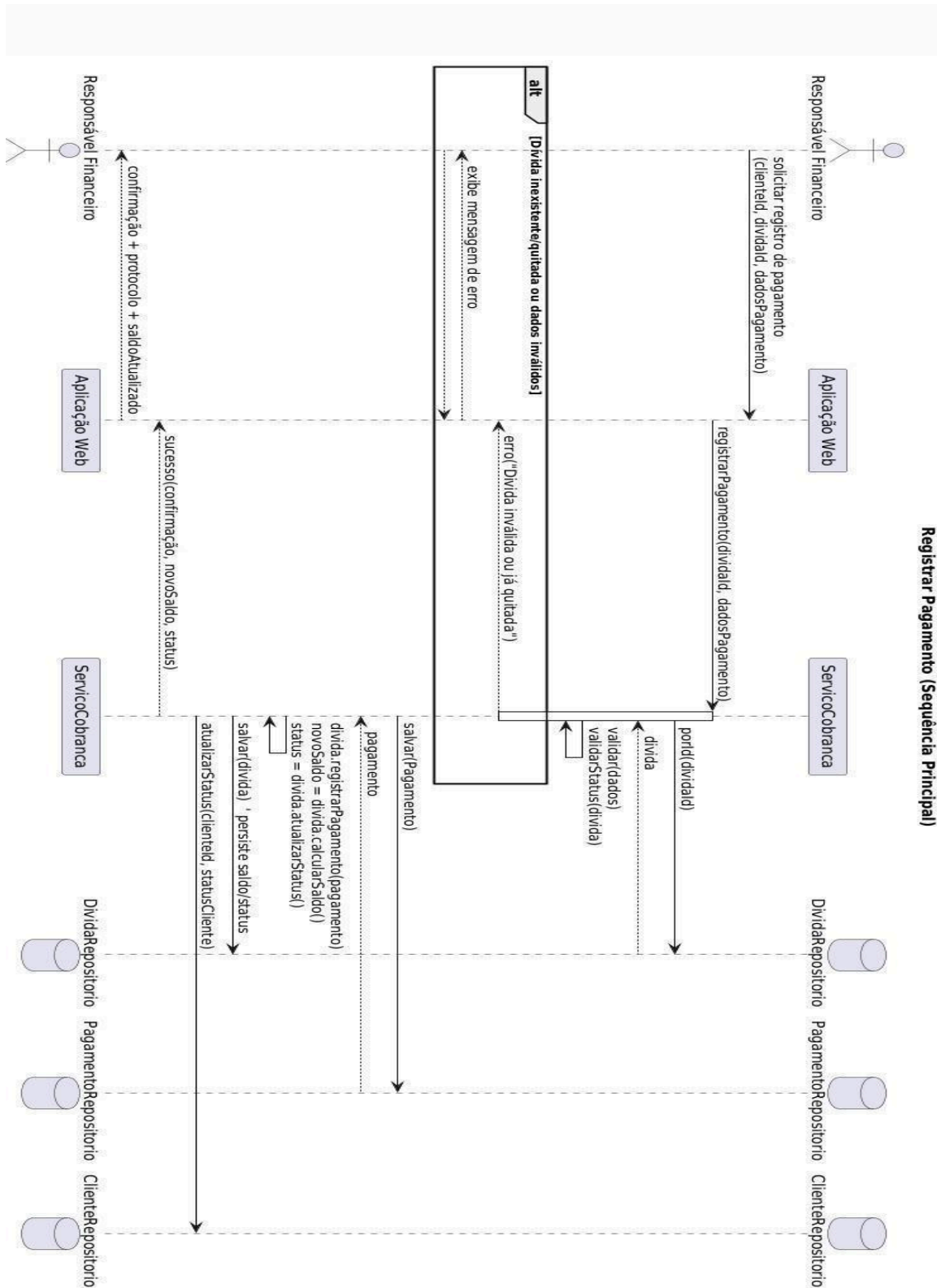


Figura 8— Diagrama de Sequência

A Figura 8 apresenta o diagrama de sequência do cadastro de dívidas no sistema de gerenciamento de inadimplentes. O processo inicia quando o Responsável Financeiro solicita o registro de uma nova dívida pela aplicação web. O sistema então valida o cliente e os dados informados. Caso haja inconsistências, é exibida uma mensagem de erro. Se estiver tudo correto, a dívida é persistida no repositório e uma confirmação de sucesso com o protocolo de registro é retornada, garantindo a integridade das informações financeiras.

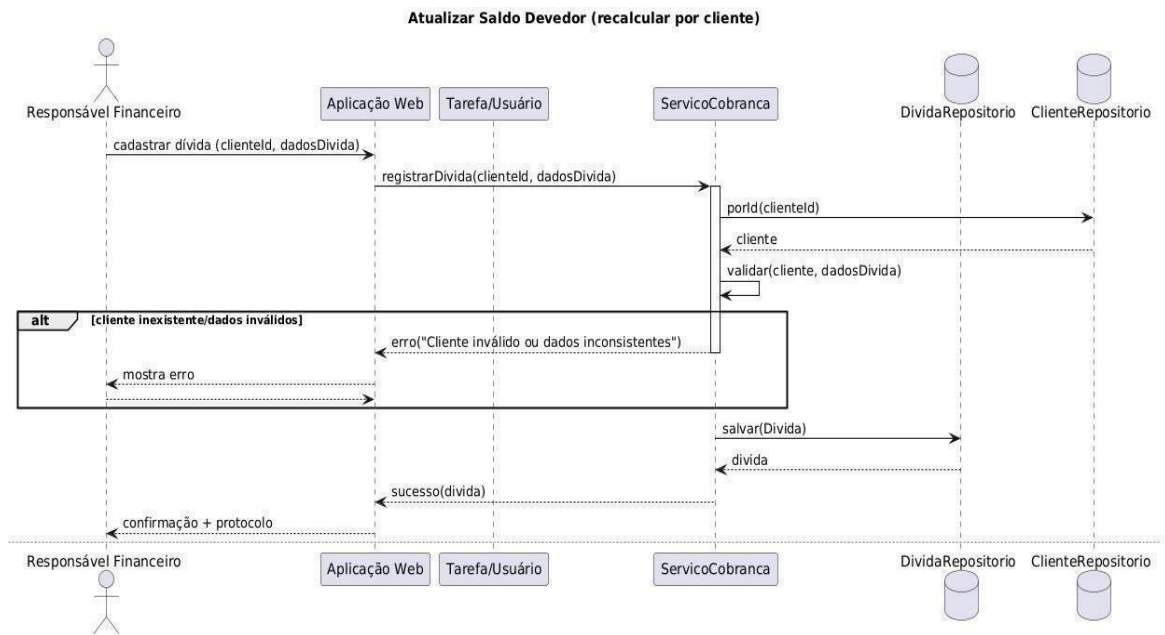


Figura 9 — Diagrama de Sequência

3.3 Diagramas de Comunicação

Nesta seção, são apresentados os diagramas de comunicação que modelam as trocas de mensagens entre atores, camadas e serviços do SGI – Sistema de Gerenciamento de Inadimplentes. O objetivo é documentar quem se comunica com quem e em que ordem, cobrindo os principais fluxos: autenticação, registro de pagamento com atualização de saldo, consulta de clientes/inadimplências e geração/expedição de relatórios. Esses diagramas complementam os diagramas de sequência e servem como referência dos contratos de mensagem da aplicação.

A Figura 10 apresenta as mensagens envolvidas no registro de pagamento de um cliente inadimplente. O Responsável Financeiro inicia o processo pela UI-Web, que valida a sessão no AuthService e encaminha a solicitação ao PagamentoService. Esse consulta o ClienteService para obter o cliente, persiste o pagamento no Repositório e, obrigatoriamente, dispara a atualização do saldo devedor (comportamento interno reutilizado). Ao final, o serviço retorna a confirmação e recibo à interface, que exibe o resultado ao usuário.

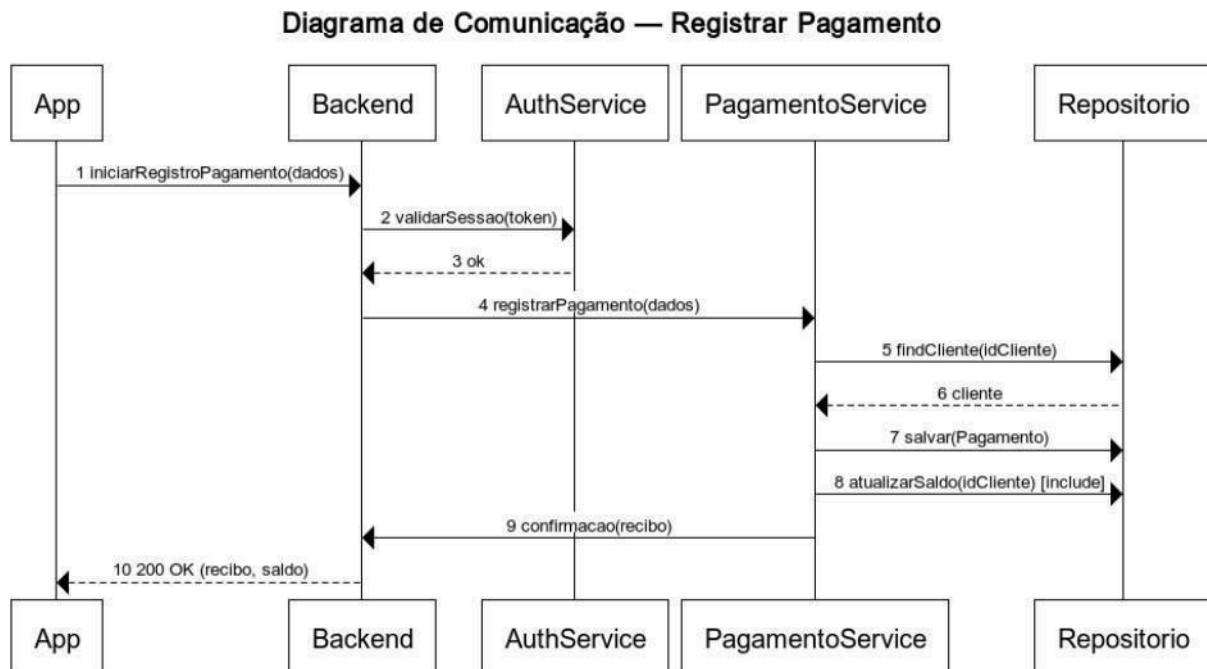


Figura 10 — Diagrama de Comunicação

Figura 11 – Diagrama de Comunicação: Gerar Relatórios (Financeiro) e Exportar (PDF/Excel)

A Figura 11 descreve o fluxo de geração de relatórios financeiros. O Responsável Financeiro solicita o relatório à UI-Web, que chama o RelatorioService. O serviço consulta dados no Repositório e devolve o conjunto resultante para exibição. Opcionalmente, quando o usuário seleciona Exportar, a interface aciona o *ExportService* para produzir o arquivo em PDF ou Excel, disponibilizando o download ao término.

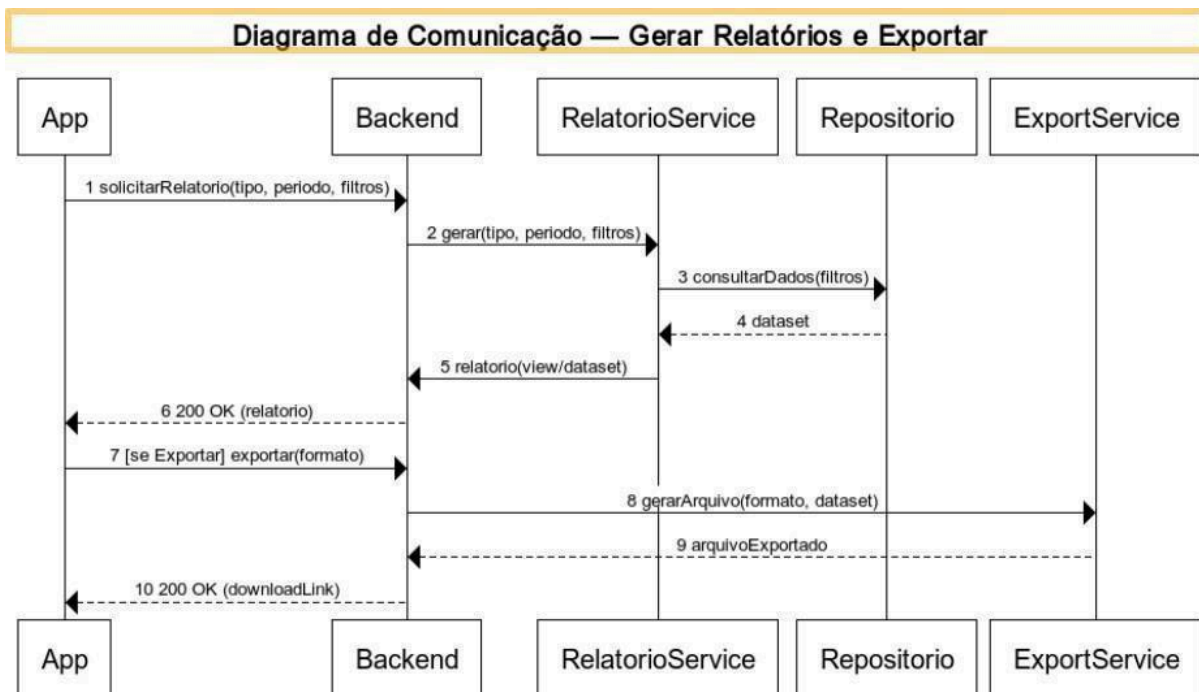


Figura 11 — Diagrama de Comunicação

Figura 12 – Diagrama de Comunicação: Consultar Cliente/Inadimplência (com cache e iteração)

A Figura 12 mostra as mensagens trocadas na consulta de clientes e suas inadimplências. O Responsável Financeiro solicita a busca à UI-Web, que encaminha ao ClienteService. Quando habilitado, o serviço consulta primeiro o Cache; em caso de cache miss, realiza query no Repositório. Em seguida, para cada cliente retornado, o serviço itera a obtenção dos detalhes de inadimplência, consolidando o resultado para exibição.

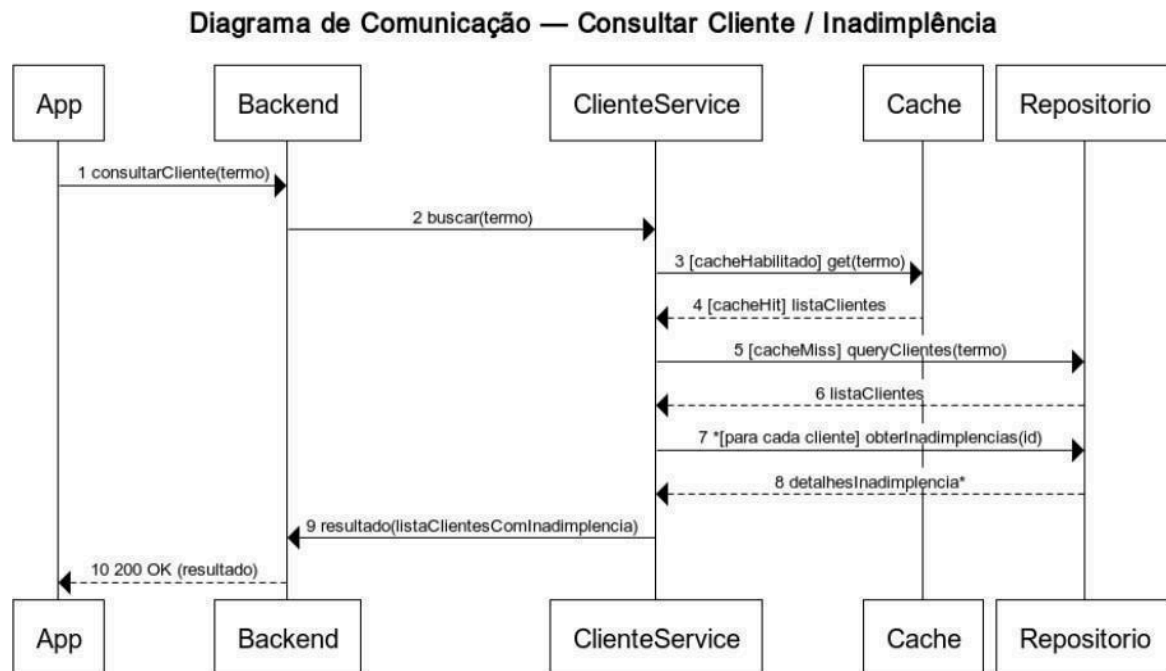


Figura 12 — Diagrama de Comunicação

3.4 Arquitetura

A arquitetura do sistema está organizada em três camadas principais — UI, API e Data — com separação explícita de responsabilidades. Na camada de UI, o repositório contempla duas frentes clientes: um Frontend Web (React + Vite) e um Aplicativo Desktop (Electron + React). Ambas as interfaces consomem a API por HTTP/JSON (REST), exibindo informações ao usuário e capturando ações como cadastro, consulta e edição de dados.

A camada de API é implementada em Spring Boot (Java 21) e concentra a orquestração de regras de negócio e a exposição dos endpoints REST. Nela, os Controllers atuam como pontos de entrada da aplicação, recebendo as requisições dos clientes, acionando a lógica de domínio e retornando respostas padronizadas. O acesso a dados é encapsulado por Repositories (Spring Data JPA), que abstraem consultas e operações CRUD. A organização em pacotes (por exemplo, um pacote de domínio como cliente contendo entidade, repository e controller) favorece coesão e facilita a evolução incremental de novas funcionalidades. Há também um pacote de configuração (ex.: CorsConfig) para políticas de CORS e demais ajustes transversais, além de integrações de suporte

como validação (Bean Validation) e documentação (springdoc/OpenAPI), que facilitam testes e inspeção dos endpoints durante o desenvolvimento.

Na camada de Data, o sistema utiliza SQLite como banco de dados local, acessado via JPA/Hibernate. As entidades modelam o domínio (por exemplo, Cliente), enquanto os Repositories executam persistência e consultas tipadas. Essa abordagem simplifica a configuração, reduz dependências externas e é adequada para cenários de implantação leve; caso a aplicação cresça em concorrência e volume, a troca por um SGDB servidor (PostgreSQL, MySQL, etc.) é direta, mantendo o mesmo contrato JPA.

Esse arranjo em UI → API → Data promove alta coesão dentro de cada módulo e baixo acoplamento entre camadas, o que melhora testabilidade, manutenibilidade e escalabilidade. Os Controllers permanecem finos, delegando persistência aos Repositories; e, à medida que as regras de negócio se tornarem mais ricas, recomenda-se introduzir uma camada de Services (ex.: ClienteService) para centralizar validações e transações, mantendo a API estável e o acesso a dados isolado. Assim, o projeto segue um padrão de arquitetura limpo e extensível, pronto para receber novos domínios (novos pacotes com Entity/Repository/Controller) sem comprometer o restante do sistema.

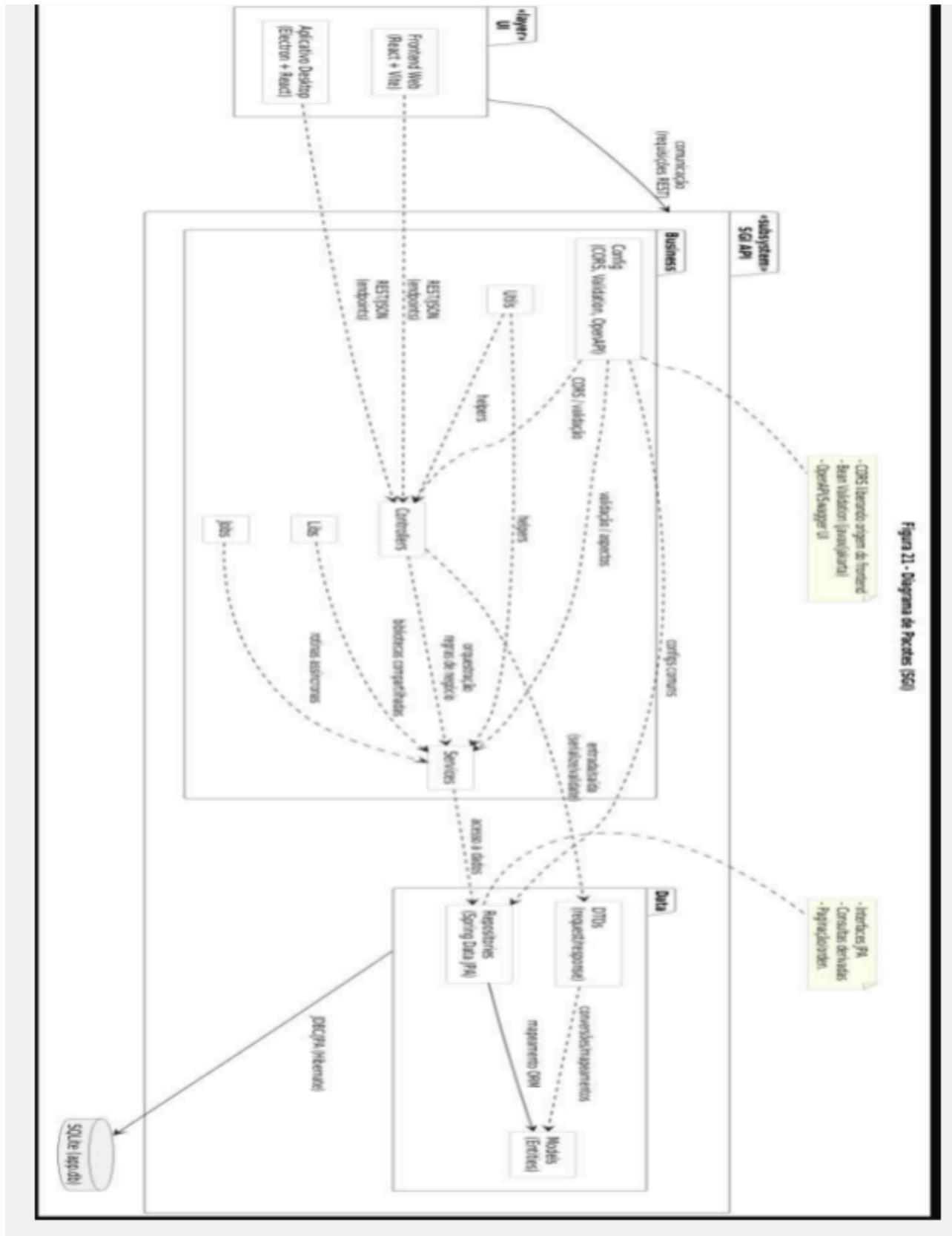


Figura 13 — Diagrama de pacote

3.5 Diagramas de Estados

O diagrama de estados modela o ciclo de vida do status de débito de um cliente no sistema, cujo escopo é apenas registrar e manter informações de inadimplência — sem ações de cobrança externa. O fluxo inicia em SemDébito, que representa o cliente cadastrado sem pendências financeiras. A partir da operação `lançar_debito()`, o estado transita para EmDébito, indicando a existência de saldo devedor. Dentro desse estado, o subestado Aberto captura a situação padrão do débito em acompanhamento, permitindo `pagamento_parcial(valor)` enquanto persistir saldo ($\text{saldo} > 0$) e evoluindo para Quitado quando ocorre `pagamento_total()` (ou quando $\text{saldo} == 0$ após ajustes). Em casos de correção operacional, `estornar_debito()` retorna o cliente a SemDébito. O ciclo pode ser encerrado a qualquer momento por `encerrar_cliente()`, levando ao estado Encerrado (inativação no sistema).

As transições foram definidas para refletir operações rotineiras e controladas: `cadastrar_cliente()` cria o registro inicial; `lançar_debito()` cria/atualiza lançamentos; `pagamento_parcial()` e `pagamento_total()` reduzem o saldo até a quitação; `ajustar_lancamento()` permite recalcular o saldo sem alterar o estado macro (EmDébito), preservando o histórico; `estornar_debito()` corrige lançamentos indevidos; `encerrar_cliente()` finaliza a relação ativa no sistema; e `novo_debito()` reabre o ciclo a partir de Quitado, retornando a SemDébito → EmDébito conforme necessário. Não há transições para eventos de protesto, negativação ou cobrança judicial, pois não fazem parte do escopo do Night Plants.

Esse modelo oferece clareza de regras, previsibilidade de transições e simplicidade de manutenção: cada estado representa uma situação operacional inequívoca (sem débito, em débito, quitado, encerrado), e cada evento corresponde a ações do usuário ou rotinas do sistema com efeitos bem definidos sobre o saldo. Além de facilitar testes e auditoria (por evidenciar quando e por que um status mudou), a máquina de estados serve como base para validações de negócio (por exemplo, impedir lançamentos em Encerrado) e para traçar métricas como tempo médio em EmDébito ou taxa de reversão por estorno, apoiando decisões de melhoria contínua.

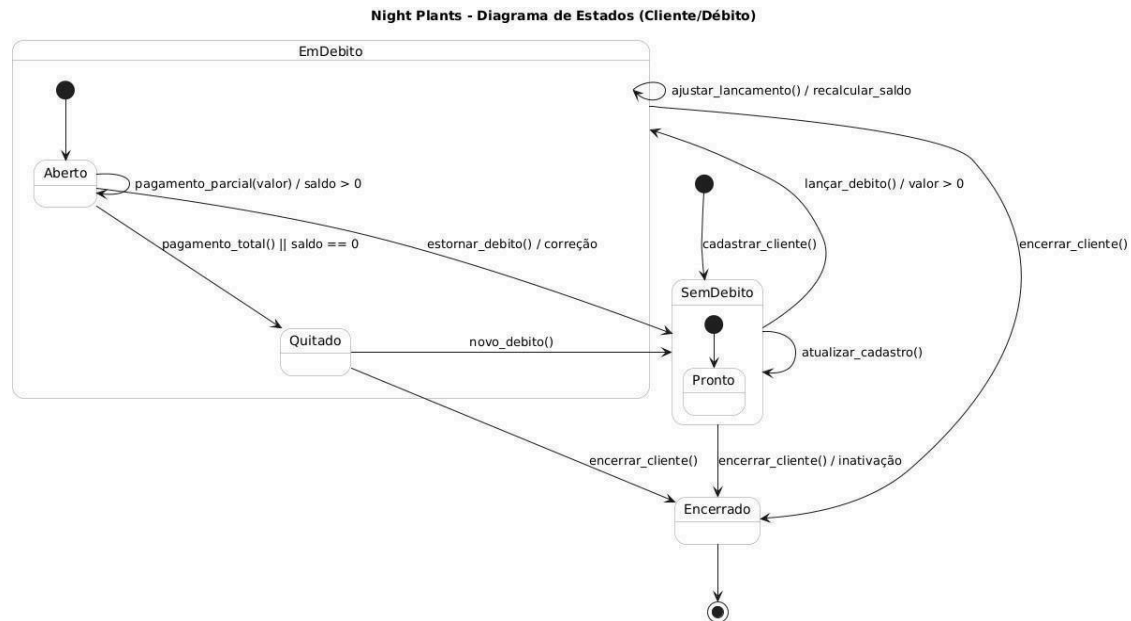


Figura 14 — Diagrama de Estado

3.6 Diagrama de Componentes e Implantação.

O diagrama de componentes apresenta a decomposição lógica do sistema em elementos independentes e suas interfaces. A solução é organizada em três blocos: UI, API e Persistência.

Na camada de UI, há dois clientes: (i) o Frontend Web (React + Vite), executado no navegador, e (ii) o Aplicativo Desktop (Electron + React). Ambos se comunicam exclusivamente com a API por HTTP/JSON (REST), realizando operações de cadastro, consulta, atualização e encerramento de registros de devedores.

A camada de API é implementada com Spring Boot (Java 21) e expõe endpoints REST por meio de Controllers (por exemplo, DevedorController). Os Controllers delegam as regras de negócio ao Service (DevedorService), que centraliza validações e fluxos (lançar débito, ajustar lançamento, quitar, encerrar). O acesso a dados é encapsulado por Repositories (DevedorRepository, Spring Data JPA), enquanto as Entities/Models (como Devedor) representam o domínio persistido. Componentes transversais (Config) tratam de CORS, validação e documentação (OpenAPI). DTOs/Mapper padronizam entrada/saída e evitam acoplamento direto da API às entidades.

Na camada de Persistência, o sistema utiliza SQLite em arquivo único (app.db), acessado via JPA/Hibernate. Essa escolha reduz dependências operacionais e atende ao escopo do Night Plants — que somente registra devedores, sem cobrança externa, protesto ou integrações financeiras. O desenho favorece alta coesão (cada componente tem uma responsabilidade clara) e baixo acoplamento (UI ↔ API ↔ Dados), simplificando manutenção, testes e evolução incremental.

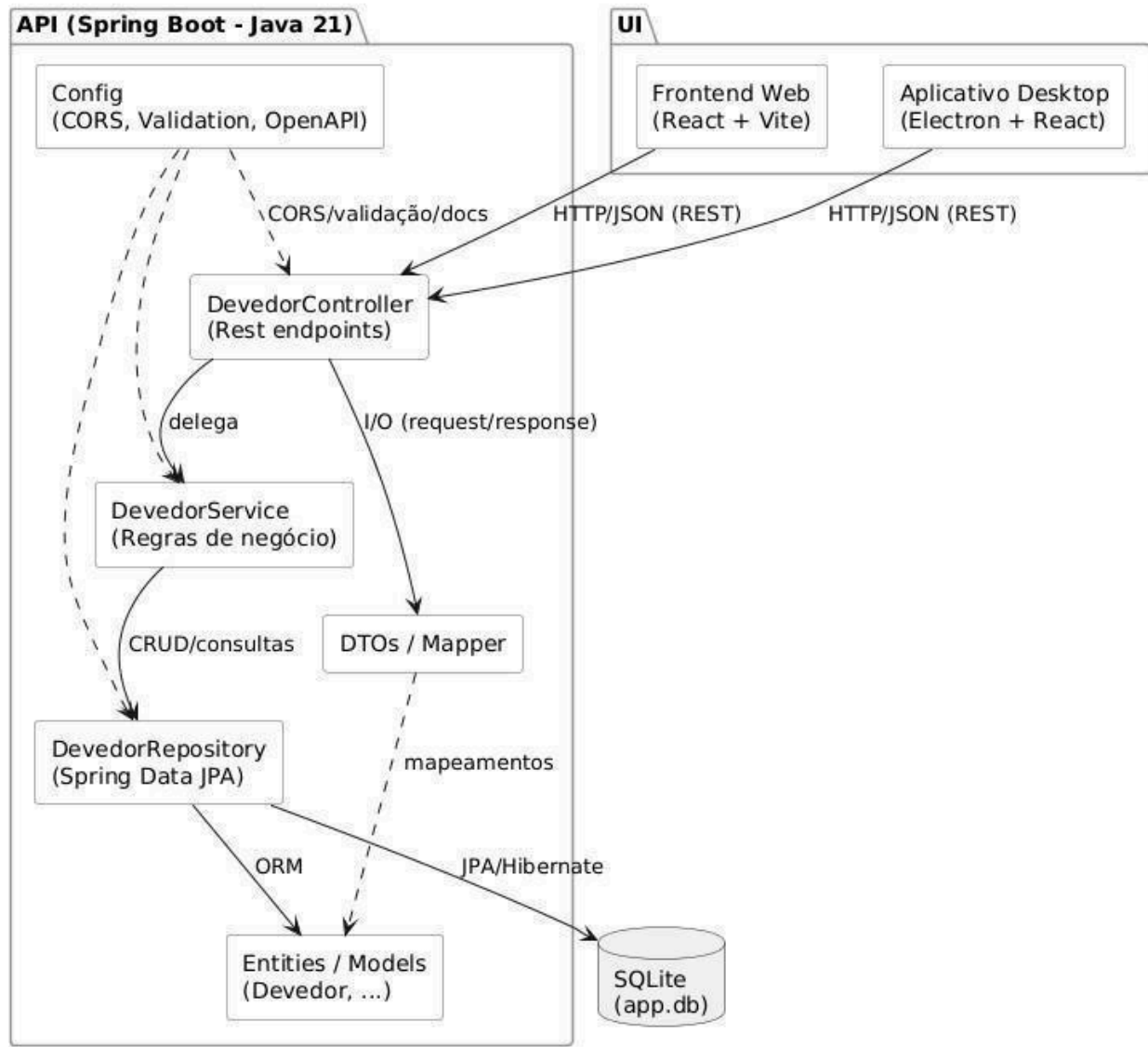


Figura 15 — Diagrama de Componentes

O diagrama de implantação descreve a distribuição física (ou virtual) dos artefatos. No cliente, o Navegador acessa o Frontend Web estático (que pode estar hospedado em um serviço de *static hosting*), enquanto o Aplicativo Desktop (Electron) roda localmente no Computador do Usuário. Ambos se conectam à API Night Plants por HTTPS.

No lado do servidor, um Servidor de Aplicação executa o artefato `nightplants.jar` (Spring Boot/Java 21). O serviço expõe a porta 8080/8443 e lê/escreve dados em um volume persistente onde reside o arquivo SQLite (`app.db`). Logs de aplicação são gravados no disco do servidor. Em produção, recomenda-se adicionar um proxy reverso (por exemplo, Nginx ou IIS) para terminação TLS, *rate limiting* e *logging* centralizado; contudo, a topologia mínima funciona com uma única instância da API e armazenamento local em SQLite, atendendo ao escopo do sistema.

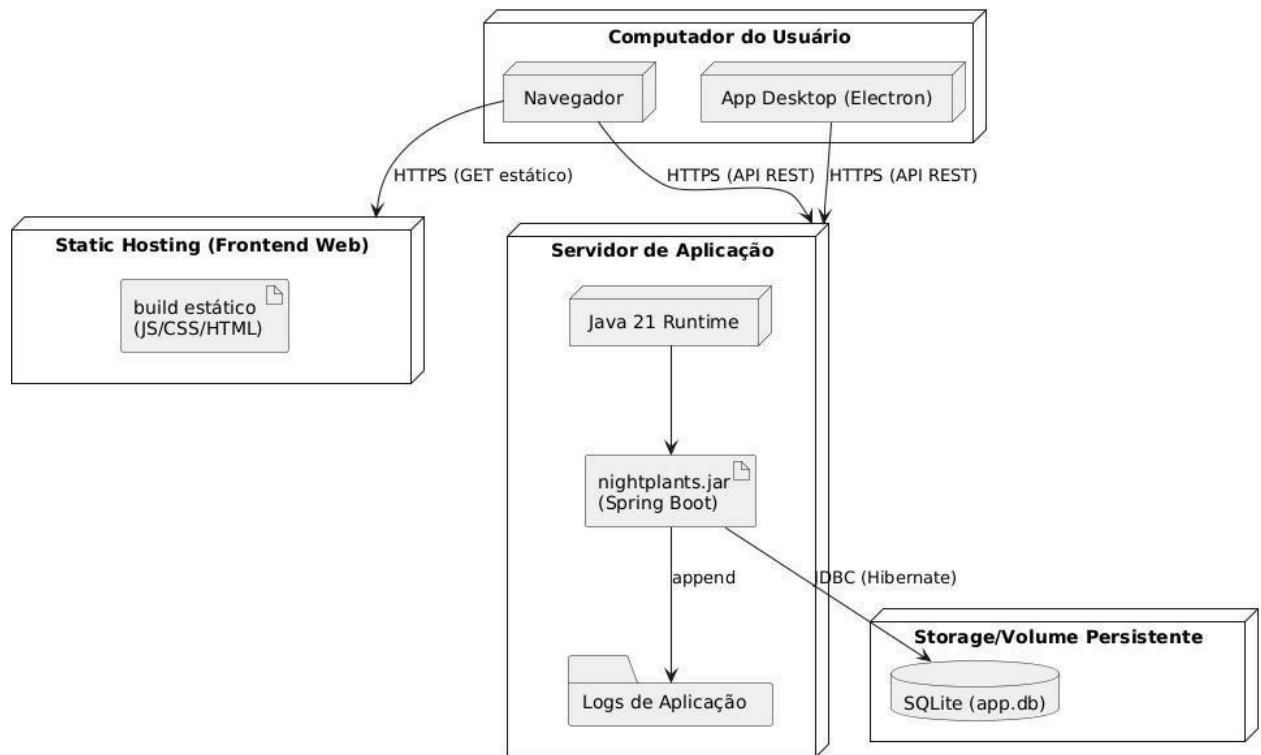


Figura 16 — Diagrama de Implantação

4. Projeto de Interface com Usuário

Esta seção tem como objetivo mostrar e descrever as interfaces de interação com o usuário das quais a aplicação é composta. Para isso foi realizado um *mockup* de baixa fidelidade usando a ferramenta paint.

4.1 Esboço das Interfaces Comuns a Todos os Atores

A tela Dashboard apresenta um resumo geral do sistema, exibindo informações como o total de clientes cadastrados, o número de inadimplentes e o valor total em aberto. Além disso, disponibiliza atalhos para o Cadastro de Cliente e para a área de Relatórios. Tanto a Cláudia quanto o José Carlos terão acesso a esta interface, permitindo que acompanhem rapidamente a situação da inadimplência e naveguem para as demais funcionalidades do sistema.

Menu

- Dashboard
- Cientes
- Inadimplentes
- Relatórios

Sistema de Gestão de Inadimplentes

Visão geral do sistema de gestão de inadimplentes

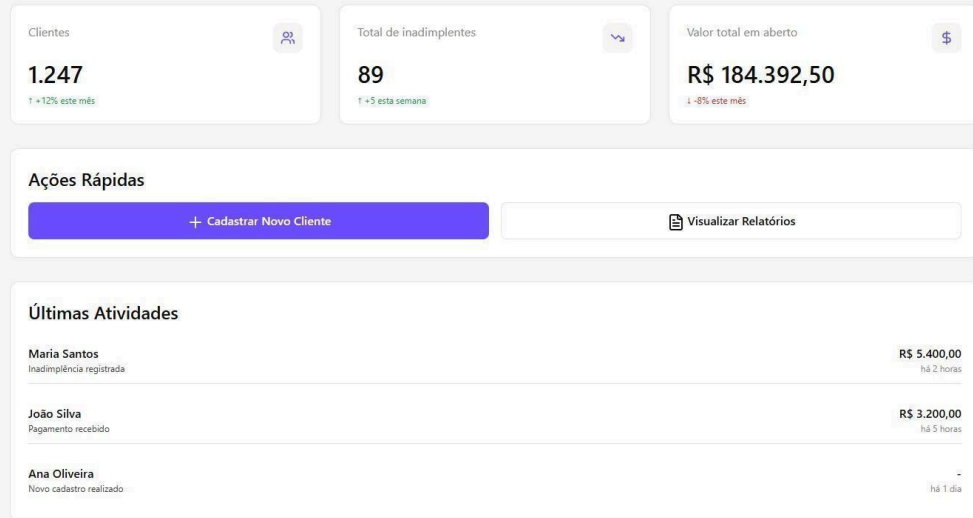


Figura 17. Tela de Dashboard

4.2 Esboço das Interfaces Usada por Ambos

A tela Cadastro de Cliente permite o registro de novos clientes no sistema, contendo campos para inserir informações essenciais como nome, CPF/CNPJ, endereço, telefone e e-mail. Após o preenchimento, é possível salvar os dados ou cancelar a operação. Tanto a Cláudia quanto o José Carlos terão acesso a esta interface, garantindo que novos clientes sejam adicionados de forma organizada e padronizada.

The 'Cadastro de Cliente' form is designed for easy data entry. It includes a sidebar menu and a main form area with labeled input fields and validation markers.

Menu

- Dashboard
- Cientes
- Inadimplentes
- Relatórios

Cadastro de Cliente

Nome * CPF/CNPJ *

Endereço

Telefone * E-mail

Salvar Cancelar

Figura 18. Tela de Cadastro de Cliente

4.3 Esboço das Interfaces Usadas por ambos

A tela Listagem de Clientes apresenta uma visão organizada de todos os clientes cadastrados no sistema, exibindo informações como nome, CPF/CNPJ e situação (adimplente ou inadimplente), e o telefone pra contato.

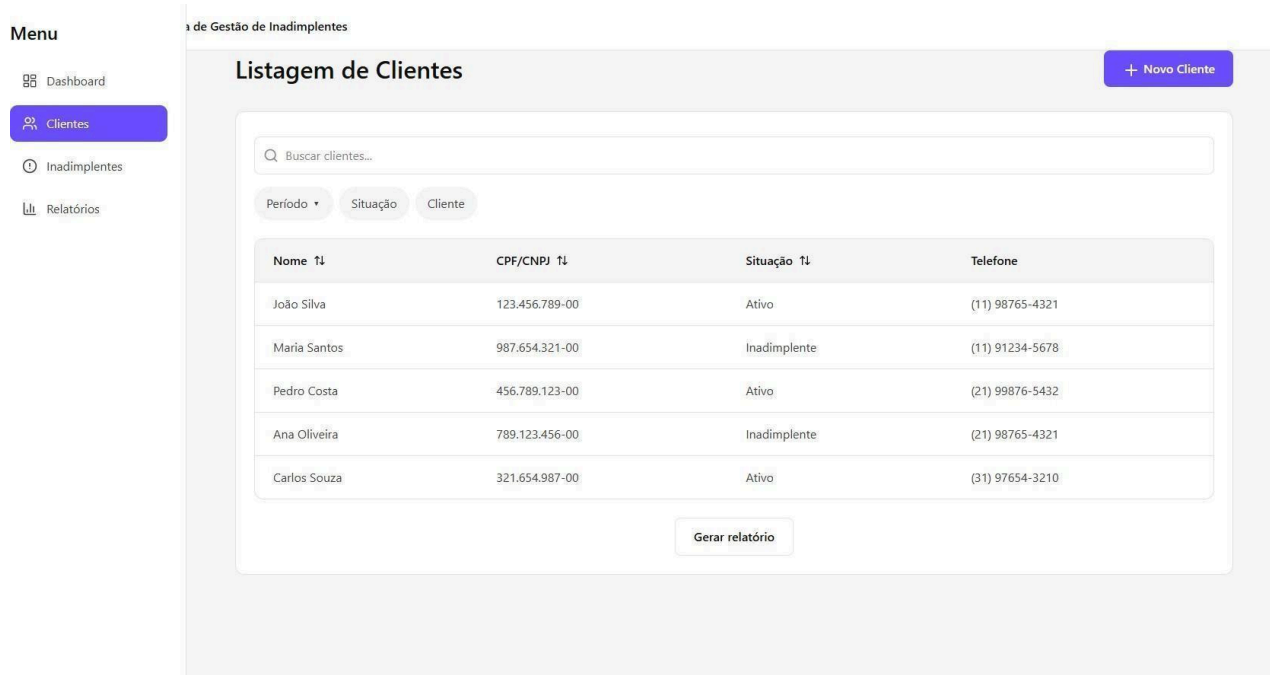


Figura 19. Tela de Listagem de Clientes

4.4 Esboço das Interfaces Usadas por ambos

A tela Lista de Inadimplentes exibe os clientes que possuem pendências financeiras no sistema. São apresentadas informações como o nome do cliente, o valor da dívida e a data de vencimento, permitindo o acompanhamento da situação de cada inadimplência. Tanto a Cláudia quanto o José Carlos terão acesso a esta interface, de forma a facilitar o controle e a tomada de medidas para regularização dos débitos.

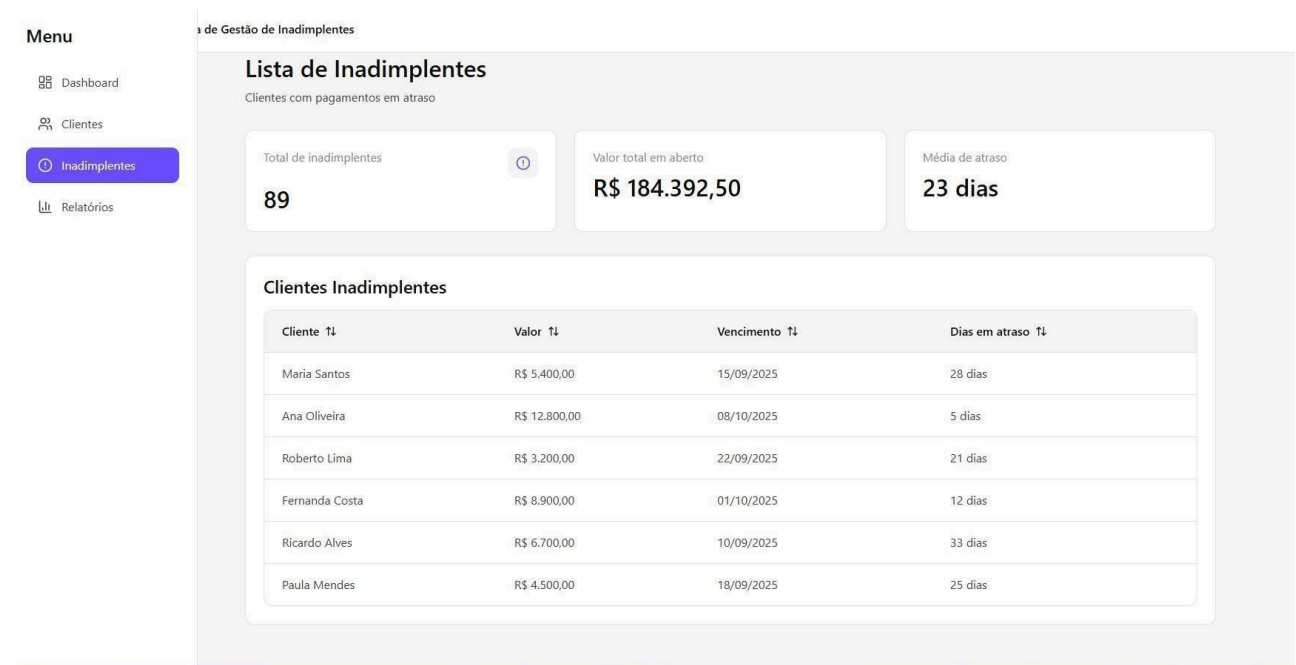


Figura 20. Tela de Lista de Inadimplentes

4.5 Esboço das Interfaces Usadas por ambos

A tela Relatórios possibilita a geração de relatórios personalizados a partir de filtros como período, cliente e status das dívidas. Além da listagem, a interface apresenta informações em formato gráfico, facilitando a análise visual dos dados. Tanto a Cláudia quanto o José Carlos terão acesso a essa funcionalidade, permitindo uma visão estratégica sobre a inadimplência e apoiando o processo de tomada de decisão.

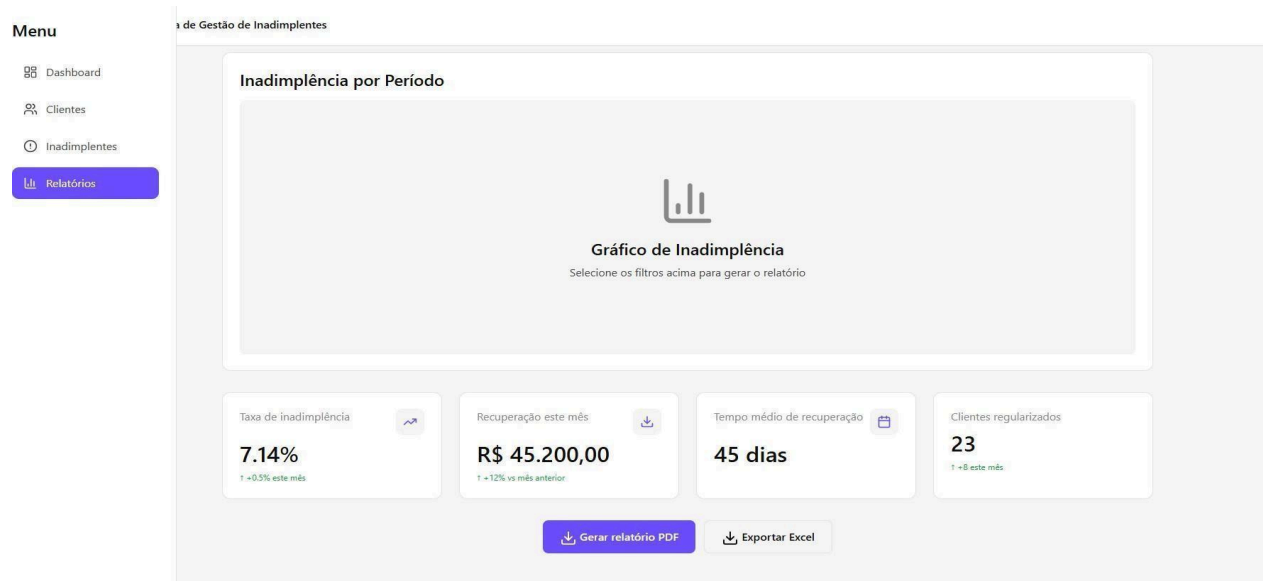


Figura 20 Tela de Relatórios

4.6 Esboço das Interfaces Usadas por ambos

A tela Registro de Inadimplência permite associar um cliente a uma dívida específica, registrando informações como o valor devido, a data de vencimento e a descrição da pendência. Essa funcionalidade garante que os débitos sejam controlados de forma detalhada e organizada. Tanto a Cláudia quanto o José Carlos terão acesso a esta interface, assegurando o correto acompanhamento das inadimplências no sistema.

Registro de Inadimplência ×

Cliente *

Selecione o cliente ▾

Valor da dívida *

R\$ 0,00

Data de vencimento *

dd/mm/aaaa

Descrição da dívida

Descreva os detalhes da inadimplência

Cancelar Salvar

Figura 21. Tela de Registro de Inadimplentes..

5. Glossário e Modelos de Dados

Esta seção descreve o glossário do sistema e o modelo de dados do SGI – Sistema de Gerenciamento de Inadimplentes. Primeiro, apresentam-se tabelas com os atributos de entrada e

saída usados nas telas e relatórios, padronizando nome, formato e significado. Em seguida, apresenta-se o modelo relacional (DER e DDL essencial) e as estratégias de mapeamento entre objetos da aplicação (camada de domínio) e tabelas do banco de dados.

ACESSO

Atributo	Formato	Descrição
Login	String	Telefone usado no login.
senha	String	Hash/salt da senha (se aplicável).
perfil	enum	Perfil de acesso
tokenacesso	String	Token
statusUsuario	Enum	Situação

Tabela 1 acesso.

TABELA CLIENTE

Atributo	Formato	Descrição
clienteId	UUID	Identificador Cliente
nome	String	Nome completo.
cpfnpj	String	Documento do cliente
email	String	email
telefone	String	Telefone para contato
endereço	String	Endereço completo
statuscliente	Enum {Ativo, Inativo}	Situação do cadastro
saldo	INTEGER	Saldo
atualizadoem	Timestamp	Ultima atualização

Tabela 2 Tabela Cliente..

TABELA RELATÓRIO E INDICADORES

Atributo	Formato	Descrição
----------	---------	-----------

rankingMaioresDevedores	Lista {clienteId, nome, saldoCentavos}	Ranking ordenado por saldo.
resumoPeriodo	{ini,fim, totalPago, totalAberto}	Indicadores por intervalo.
exportArquivo	PDF/Excel	Resultado exportado sob demanda.

Tabela 3 relatório e indicadores.

TABELA PAGAMENTO

Atributo	Formato	Descrição
pagamentoId	UUID/API • INTEGER/BD	Identificador do pagamento.
clienteId	FK → Cliente	Cliente vinculado.
valorPago	INTEGER	Valor pago
dataPagamento	Date	Data do pagamento.
comprovante	Blob/URL	Comprovante (opcional).
criadoEm	Timestamp	Registro do lançamento.

Tabela 4 Pagamentos

TABELA INADIMPLÊNCIA

Atributo	Formato	Descrição
inadimplenciaId	UUID/API • INTEGER/BD	Identificador da dívida.
clienteId	FK → Cliente	Cliente associado.
valorDevidoCentavos	INTEGER	Valor da dívida (centavos).
vencimento	Date	Data de vencimento.
descricao	String	Observações (opcional).

statusInadimplencia	Enum	Estado da dívida.
criadoEm	Timestamp	Momento do registro.

Tabela 5 Tabela Inadimplencia..

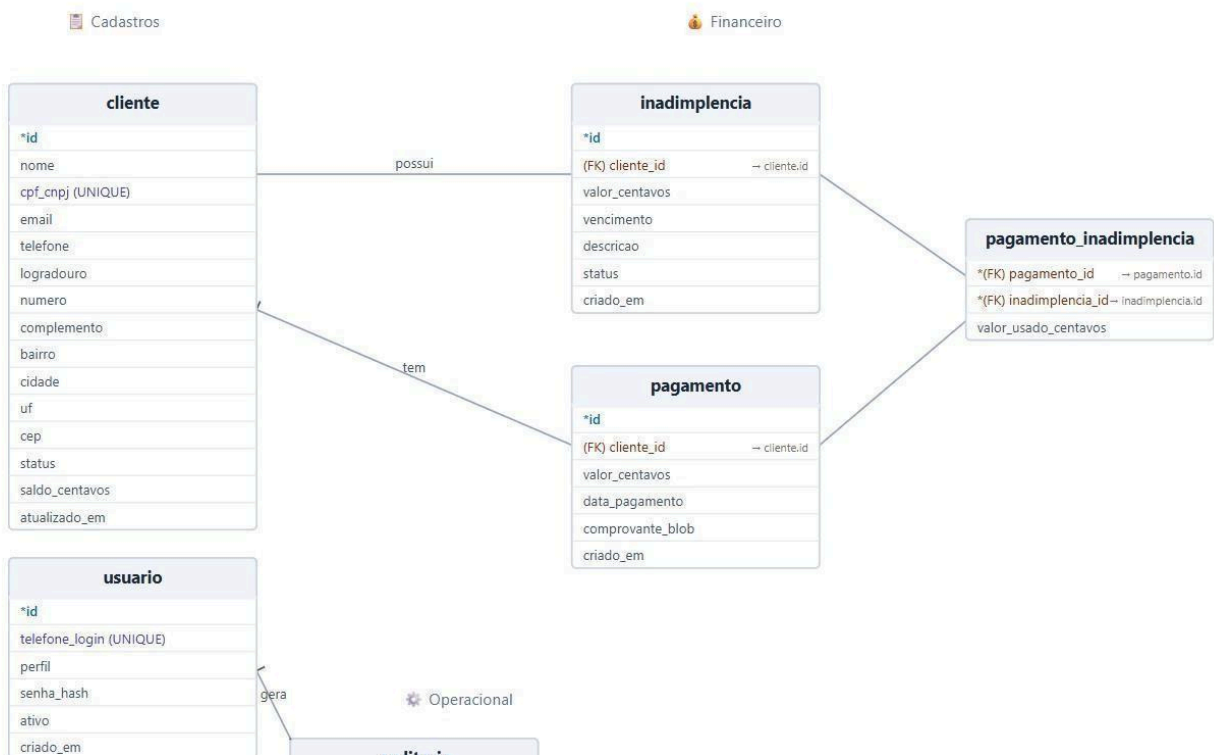


Figura 21. Diagrama de entidade e relacionamento

6. Casos de Teste

O Sistema de Gerenciamento de Inadimplentes (SGI) será validado principalmente por meio de testes funcionais de caixa-preta, executados manualmente em ambiente de homologação com a mesma configuração prevista para produção. Os testes foram derivados dos casos de uso, das histórias de usuário (US1 a US10) e dos contratos de operação, contemplando autenticação, cadastro de clientes, registro de inadimplências, registro de pagamentos, atualização de saldo e geração de relatórios. Além dos testes funcionais, serão observados requisitos não funcionais básicos, como usabilidade (telas claras e coerentes com o trabalho diário do escritório), desempenho adequado nas operações mais comuns e segurança de acesso às informações. O sistema será considerado aceito quando os casos de teste críticos (login, cadastro de cliente, registro de inadimplência, pagamento, recálculo de saldo e relatórios principais) estiverem aprovados e não houver defeitos bloqueadores nas principais funcionalidades.

6.1 Plano de Teste de Aceitação

Os testes de aceitação têm como objetivo verificar se o SGI atende às principais necessidades do escritório de contabilidade, alinhadas ao Documento de Visão e às histórias de usuário. Para fins de organização, foram definidas cinco necessidades principais, e para cada uma delas são apresentados três casos de teste de aceitação.

- **Necessidade 1** – Garantir acesso seguro ao sistema (autenticação e perfis).
- **Necessidade 2** – Manter o cadastro de clientes atualizado e consistente.
- **Necessidade 3** – Registrar inadimplências dos clientes.
- **Necessidade 4** – Registrar pagamentos e atualizar automaticamente o saldo devedor.
- **Necessidade 5** – Disponibilizar relatórios e indicadores para apoio à gestão.

O identificador dos casos de teste segue o padrão T<número do teste>A<número da necessidade>. Exemplo: T1A1 corresponde ao primeiro teste da necessidade 1.

6.1.1 Necessidade 1 – Acesso seguro ao sistema

Caso de Teste T1A1 – Login com credenciais válidas

Campo	Descrição
Identificador	T1A1
Caso de teste	Realizar login com credenciais válidas e acessar o sistema conforme o perfil.
Pré-condições	Usuário cadastrado, ativo no sistema; telefone/senha corretos; aplicação Web ou Desktop em execução.
Ações	(1) Abrir a tela de login. (2) Informar telefone e senha válidos. (3) Acionar o botão "Entrar".
Resultados esperados	A autenticação é realizada com sucesso; o sistema gera um token de sessão; o usuário é redirecionado para o dashboard inicial de acordo com seu perfil.

Tabela 6 Tabela Caso de Teste T1A1.

Caso de Teste T2A1 – Login com senha inválida

Campo	Descrição
Identificador	T2A1
Caso de teste	Tentar realizar login com senha inválida.
Pré-condições	Usuário cadastrado e ativo; telefone correto; senha incorreta.
Ações	(1) Abrir a tela de login. (2) Informar telefone válido e senha incorreta. (3) Acionar o botão "Entrar".
Resultados esperados	O sistema recusa o acesso, não gera token de sessão e exibe mensagem de erro informando credenciais inválidas, mantendo o usuário na tela de login.

*Tabela 7 Tabela Caso de Teste T2A1.***Caso de Teste T3A1 – Acesso sem autenticação a área restrita**

Campo	Descrição
Identificador	T3A1
Caso de teste	Tentar acessar diretamente uma tela restrita sem estar autenticado.
Pré-condições	Nenhuma sessão autenticada ativa; URL de tela restrita conhecida (por exemplo, lista de inadimplentes).
Ações	(1) Tentar acessar a URL de uma tela restrita digitando o endereço direto no navegador ou abrindo o app sem login.
Resultados esperados	O sistema impede o acesso e redireciona o usuário para a tela de login, exibindo mensagem informando a necessidade de autenticação.

*Tabela 8 Tabela Caso de Teste T3A1.***6.1.2 Necessidade 2 – Cadastro e manutenção de clientes****Caso de Teste T1A2 – Cadastrar novo cliente com dados válidos**

Campo	Descrição
Identificador	T1A2
Caso de teste	Cadastrar um novo cliente com todas as informações obrigatórias preenchidas corretamente.
Pré-condições	Usuário autenticado com permissão de cadastro; conexão

	com o banco de dados ativa.
Ações	(1) Acessar a tela "Cadastro de Cliente". (2) Preencher nome, CPF/CNPJ, endereço, telefone e e-mail válidos. (3) Acionar o botão "Salvar".
Resultados esperados	O sistema valida os dados, salva o novo cliente na base, exibe mensagem de sucesso e o cliente passa a aparecer na lista de clientes cadastrados.

Tabela 9 Tabela Caso de Teste T1A2.

Caso de Teste T2A2 – Impedir cadastro com CPF/CNPJ duplicado

Campo	Descrição
Identificador	T2A2
Caso de teste	Impedir o cadastro de um cliente com CPF/CNPJ já existente.
Pré-condições	Cliente já cadastrado com determinado CPF/CNPJ; usuário autenticado com permissão de cadastro.
Ações	(1) Acessar a tela "Cadastro de Cliente". (2) Informar um CPF/CNPJ já existente. (3) Preencher demais campos e acionar "Salvar".
Resultados esperados	O sistema identifica o CPF/CNPJ duplicado, não grava o cadastro e exibe mensagem de erro informando que já existe cliente com esse documento.

Tabela 10 Tabela Caso de Teste T2A2

Caso de Teste T3A2 – Atualizar dados de cliente existente

Campo	Descrição
Identificador	T3A2
Caso de teste	Editar os dados de um cliente já cadastrado.
Pré-condições	Cliente existente na base; usuário autenticado.
Ações	(1) Acessar a tela "Listagem de Clientes". (2) Selecionar um cliente. (3) Acionar a opção "Editar". (4) Alterar endereço/telefone. (5) Salvar alterações.

Resultados esperados	As alterações são validadas e salvas; o sistema exibe mensagem de sucesso; a lista de clientes e os detalhes do cliente são atualizados com os novos dados.
----------------------	---

Tabela 11 Tabela Caso de Teste T3A2.

6.1.3 Necessidade 3 – Registro de inadimplências

Caso de Teste T1A3 – Registrar nova inadimplência

Campo	Descrição
Identificador	T1A3
Caso de teste	Registrar uma nova inadimplência para um cliente ativo.
Pré-condições	Cliente cadastrado e ativo; usuário autenticado; tela de registro de inadimplência acessível.
Ações	(1) Acessar a tela "Registro de Inadimplência". (2) Selecionar o cliente. (3) Informar valor devido, data de vencimento e descrição. (4) Acionar "Salvar".
Resultados esperados	A inadimplência é validada e registrada com status inicial "Em Aberto"; o sistema gera um identificador/protocolo e exibe mensagem de sucesso.

Tabela 12 Tabela Caso de Teste T1A3.

Caso de Teste T2A3 – Impedir registro com dados inválidos

Campo	Descrição
Identificador	T2A3
Caso de teste	Impedir o registro de inadimplência com valor inválido ou cliente inexistente.
Pré-condições	Usuário autenticado.
Ações	(1) Acessar "Registro de Inadimplência". (2) Informar valor ≤ 0 ou selecionar um cliente inexistente (ou removido). (3) Acionar "Salvar".
Resultados esperados	O sistema valida os campos, não grava a inadimplência e exibe mensagens de erro indicando o problema (valor inválido ou cliente não encontrado).

Tabela 13 Tabela Caso de Teste T2A3.

Caso de Teste T3A3 – Listar inadimplências por cliente

Campo	Descrição
Identificador	T3A3
Caso de teste	Exibir a lista de inadimplências filtrada por cliente.
Pré-condições	Cliente com uma ou mais inadimplências registradas; usuário autenticado.
Ações	(1) Acessar a tela "Lista de Inadimplentes". (2) Aplicar filtro pelo cliente selecionado.
Resultados esperados	O sistema apresenta apenas as inadimplências vinculadas ao cliente filtrado, exibindo valor, vencimento e status de cada registro.

Tabela 14 Tabela Caso de Teste T3A3.

6.1.4 Necessidade 4 – Registro de pagamentos e atualização de saldo**Caso de Teste T1A4 – Registrar pagamento parcial e recalcular saldo**

Campo	Descrição
Identificador	T1A4
Caso de teste	Registrar um pagamento parcial para uma dívida em aberto, recalculando o saldo devedor.
Pré-condições	Cliente com inadimplência em status "Em Aberto"; usuário autenticado; valor devido maior que o valor a pagar.
Ações	(1) Acessar tela de "Registro de Pagamento". (2) Selecionar o cliente e a dívida. (3) Informar valor de pagamento menor que o saldo total. (4) Confirmar operação.
Resultados esperados	O pagamento é registrado; o saldo da dívida é recalculado e permanece em aberto; o status da dívida é atualizado (por exemplo, "Parcial"); o novo saldo é exibido ao usuário.

Tabela 15 Tabela Caso de Teste T1A4.

Caso de Teste T2A4 – Quitar dívida e atualizar status do cliente

Campo	Descrição
Identificador	T2A4
Caso de teste	Registrar pagamento que quita a dívida e atualizar o status do cliente.
Pré-condições	Cliente com uma única dívida em aberto; usuário autenticado; valor de pagamento igual ao saldo devedor.
Ações	(1) Acessar "Registro de Pagamento". (2) Selecionar o cliente e a dívida. (3) Informar valor igual ao saldo devedor. (4) Confirmar pagamento.
Resultados esperados	A dívida é marcada como quitada; o saldo é recalculado para zero; o status do cliente é atualizado para "Sem Débito/Adimplente"; recibo/registro é exibido.

Tabela 16 Tabela Caso de Teste T2A4.

Caso de Teste T3A4 – Impedir pagamento inválido

Campo	Descrição
Identificador	T3A4
Caso de teste	Impedir o registro de pagamento com valor maior que o saldo ou para cliente inexistente.
Pré-condições	Usuário autenticado.
Ações	(1) Acessar "Registro de Pagamento". (2) Selecionar cliente/dívida ou informar dados incorretos. (3) Informar valor maior que o saldo ou apontar cliente inexistente.
Resultados esperados	O sistema valida os dados, não registra o pagamento e apresenta mensagens de erro adequadas (valor maior que o saldo, cliente/dívida não encontrados).

Tabela 17 Tabela Caso de Teste T3A4.

6.1.5 Necessidade 5 – Relatórios e indicadores

Caso de Teste T1A5 – Gerar relatório de inadimplentes por período

Campo	Descrição
Identificador	T1A5
Caso de teste	Gerar relatório de inadimplentes filtrado por período de vencimento.
Pré-condições	Usuário autenticado; existência de inadimplências registradas em diferentes períodos
Ações	(1) Acessar a tela "Relatórios". (2) Informar período inicial e final. (3) Acionar a geração do relatório.
Resultados esperados	O sistema consulta os dados e exibe relatório com clientes inadimplentes dentro do período, com informações de valor, vencimento e total em aberto.

Tabela 17 Tabela Caso de Teste T1A5.

Caso de Teste T2A5 – Exportar relatório em PDF/Excel

Campo	Descrição
Identificador	T2A5
Caso de teste	Exportar um relatório gerado para PDF ou Excel.
Pré-condições	Relatório já gerado na tela; usuário autenticado.
Ações	(1) Na tela de relatórios, após gerar o relatório, acionar a opção "Exportar". (2) Selecionar o formato (PDF ou Excel).
Resultados esperados	O sistema gera o arquivo no formato escolhido e disponibiliza para download; o arquivo contém os mesmos dados apresentados na tela.

Tabela 18 Tabela Caso de Teste T2A5.

Caso de Teste T3A5 – Visualizar dashboard com indicadores

Campo	Descrição
Identificador	T3A5

Caso de teste	Visualizar dashboard com indicadores de inadimplência (total em aberto, número de inadimplentes, ranking de maiores devedores).
Pré-condições	Usuário autenticado; existência de clientes e inadimplências no sistema.
Ações	(1) Acessar a tela "Dashboard".
Resultados esperados	O sistema apresenta os indicadores consolidados (totais, gráficos, ranking), atualizados a partir dos dados do banco, permitindo visão rápida da situação financeira.

Tabela 20 Tabela Caso de Teste T3A5.

6.2 Plano de Teste de Integração

Os testes de integração têm como objetivo verificar se os componentes do SGI interagem corretamente entre si e com recursos externos, como a API, a camada de persistência (SQLite) e as diferentes interfaces cliente (Frontend Web e Aplicativo Desktop).

A seguir são apresentados casos de teste de integração que cobrem os fluxos principais: autenticação, cadastro de clientes, registro de inadimplência, registros de pagamentos com recálculo de saldo e geração/exportação de relatórios.

Caso de Teste TI1 – Integração UI ↔ API ↔ Banco de Dados na autenticação

Campo	Descrição
Identificador	TI1
Caso de teste	Verificar integração entre interface (Web/Desktop), API de autenticação e banco de dados ao realizar login.
Pré-condições	Usuário cadastrado e ativo na base (tabela de acesso); API em execução; banco de dados SQLite acessível; frontend configurado apontando para a URL da API.
Ações	(1) No frontend, informar telefone e senha válidos. (2) Enviar requisição de login para a API. (3) API consultar repositório de usuários no banco.
Resultados esperados	A API valida as credenciais contra o banco e retorna resposta HTTP 200 com token e perfil; o frontend armazena o token (sessão) e redireciona para o dashboard.

Tabela 21 Tabela Caso de Teste TI1

Caso de Teste TI2 – Integração no cadastro de clientes

Campo	Descrição
Identificador	TI2
Caso de teste	Verificar o fluxo integrado de cadastro de cliente, do frontend até a persistência em banco.
Pré-condições	Usuário autenticado; API em execução; tabela de clientes criada; conexão com o banco de dados funcionando.
Ações	(1) No frontend, abrir a tela "Cadastro de Cliente". (2) Preencher dados válidos. (3) Enviar requisição de cadastro para a API. (4) API chamar o Service/Repository.
Resultados esperados	A API valida os dados e persiste o cliente via Repository; o banco grava o registro; a resposta HTTP de sucesso é enviada ao frontend, que exibe mensagem e atualiza a listagem.

Tabela 22 Tabela Caso de Teste TI2.

Caso de Teste TI3 – Integração no registro de inadimplência

Campo	Descrição
Identificador	TI3
Caso de teste	Verificar a integração entre UI, API e banco no registro de inadimplência vinculada a um cliente
Pré-condições	Cliente existente na base; usuário autenticado; API e banco em execução.
Ações	(1) Na UI, acessar "Registro de Inadimplência". (2) Selecionar cliente e informar valor, vencimento e descrição. (3) Enviar requisição para a API.
Resultados esperados	A API aciona o serviço de inadimplência, valida dados, grava o registro no banco e retorna protocolo/ID; a UI exibe confirmação e a inadimplência passa a aparecer na lista do cliente.

Tabela 23 Tabela Caso de Teste TI3

Caso de Teste TI4 – Integração no registro de pagamento com recálculo de saldo

Campo	Descrição
Identificador	TI4
Caso de teste	Verificar se o registro de pagamento integra corretamente UI, API, serviços de domínio e banco, atualizando o saldo devedor.
Pré-condições	Cliente com inadimplência em aberto; usuário autenticado; API e banco em execução; serviço de recálculo de saldo implementado.
Ações	(1) Na UI, acessar "Registro de Pagamento". (2) Selecionar cliente e inadimplência. (3) Informar valor de pagamento. (4) Enviar requisição à API.
Resultados esperados	A API registra o pagamento, aciona o serviço de recálculo de saldo, atualiza o registro de inadimplência e o saldo do cliente no banco; em seguida retorna novo saldo e status; a UI exibe os valores atualizados.

Tabela 24 Tabela Caso de Teste TI4.

Caso de Teste TI5 – Integração na geração e exportação de relatórios

Campo	Descrição
Identificador	TI5
Caso de teste	Verificar a integração entre UI, API, camada de relatório e banco de dados na geração e exportação de relatórios.
Pré-condições	Dados de clientes, inadimplências e pagamentos já cadastrados; usuário autenticado; API e banco operacionais.
Ações	(1) Na UI, acessar "Relatórios". (2) Informar filtros de período/cliente/status. (3) Enviar requisição para geração de relatório. (4) Após exibição, acionar "Exportar".

Resultados esperados	A API consulta o banco, consolida os dados e retorna dataset para a UI; a UI exibe o relatório; ao exportar, a API gera arquivo PDF/Excel e devolve para download, mantendo consistência com os dados exibidos.
----------------------	---

Tabela 25 Tabela Caso de Teste T15.

7. Cronograma e Processo de Implementação

O desenvolvimento do Sistema de Gerenciamento de Inadimplentes (SGI) adotará um processo incremental e iterativo, inspirado em práticas ágeis. As funcionalidades serão organizadas em histórias de usuário e distribuídas em ciclos de trabalho quinzenais, priorizando primeiro o núcleo do sistema (cadastro de clientes, registro de inadimplências, pagamentos e relatórios principais). Cada ciclo envolverá atividades de planejamento (seleção das histórias e definição de critérios de aceitação), implementação (backend em Spring Boot, banco de dados e interfaces), revisão de código e execução de testes. Como artefatos principais, serão produzidos e continuamente atualizados: Documento de Visão, backlog de requisitos, modelos UML (casos de uso, sequência, comunicação e arquitetura), modelo de dados, código-fonte versionado, evidências de testes e versões executáveis do sistema para homologação.

Para apoiar esse processo, será utilizado Git como sistema de controle de versão e um repositório remoto (por exemplo, GitHub) para centralizar o código, a documentação e o histórico de alterações. A gerência de projeto será feita por meio de um quadro kanban (GitHub Projects, Trello ou similar), com tarefas organizadas em colunas como “A Fazer”, “Em andamento” e “Concluído”, permitindo acompanhar o progresso por quinzena. Será seguido um fluxo de branches simples (branch principal e branches de funcionalidade) com code review via *pull requests*. Um pipeline básico de CI/CD (por exemplo, GitHub Actions) será configurado para compilar o projeto, executar testes automatizados (testes de unidade e de integração dos serviços principais) e sinalizar falhas antes da integração na branch principal, reduzindo riscos de regressão. Sempre que possível, serão utilizadas ferramentas de qualidade de código (como linters e análise estática) e scripts de migração/carga de banco de dados, de forma que o processo de implementação produza não apenas o sistema em funcionamento, mas também um conjunto consistente de artefatos que documentem e respaldem a evolução do SGI.

Período (2026)	Atividades previstas
1ª quinzena de fevereiro	Revisar requisitos e Documento de Visão; revisar modelos (casos de uso, diagramas, modelo de dados).
2ª quinzena de fevereiro	Ajustar diagramas conforme feedback; detalhar backlog de histórias de usuário e priorizar funcionalidades.
1ª quinzena de março	Implementar infraestrutura do backend (projeto Spring Boot, banco de dados) e serviços básicos de autenticação.
2ª quinzena de março	Implementar funcionalidades de cadastro de clientes e regras básicas de negócio no backend.
1ª quinzena de abril	Implementar registro de inadimplências e atualização de saldo na API; criar scripts de carga de dados de teste.

2ª quinzena de abril	Desenvolver interfaces web/desktop (telas de login, clientes e inadimplências) e integrar com a API.
1ª quinzena de maio	Implementar funcionalidades de pagamentos, recálculo de saldo e relatórios principais (ex.: ranking de devedores).
2ª quinzena de maio	Realizar testes funcionais e de integração; corrigir defeitos; refinar usabilidade e mensagens da interface.
1ª quinzena de junho	Consolidar evidências de testes; revisar e atualizar o Documento de Projeto com base no sistema implementado.
2ª quinzena de junho	Finalizar Documento de Projeto, revisar normas e formatação; preparar e ensaiar apresentação.

Tabela 26- Cronograma